



US012413470B2

(12) **United States Patent**
Wang et al.

(10) **Patent No.:** **US 12,413,470 B2**

(45) **Date of Patent:** ***Sep. 9, 2025**

(54) **VIRTUAL NETWORK FUNCTION
CREATION SYSTEM**

(56) **References Cited**

U.S. PATENT DOCUMENTS

(71) Applicant: **AT&T Intellectual Property I, L.P.**,
Atlanta, GA (US)

9,813,335 B2 11/2017 Felstaine et al.

9,853,914 B1 12/2017 Felstaine et al.

(Continued)

(72) Inventors: **Chengwei Wang**, Bridgewater, NJ
(US); **Oliver Spatscheck**, Denison, TX
(US)

FOREIGN PATENT DOCUMENTS

(73) Assignee: **AT&T Intellectual Property I, L.P.**,
Atlanta, GA (US)

CN 101939729 A * 1/2011 G06F 9/5077

EP 3253005 A4 1/2018

(Continued)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 252 days.

OTHER PUBLICATIONS

This patent is subject to a terminal dis-
claimer.

Chen, Yuling et al., "Realizing Network Function Virtualization
Management and Orchestration with Model Based Open Architec-
ture", IEEE 11th Int'l Network and Service Management, 2015, 9
pages.

(Continued)

(21) Appl. No.: **17/852,478**

(22) Filed: **Jun. 29, 2022**

Primary Examiner — Younes Naji

(74) *Attorney, Agent, or Firm* — Guntin & Gust, PLC;
Ralph Tremontozzi

(65) **Prior Publication Data**

US 2022/0329483 A1 Oct. 13, 2022

(57) **ABSTRACT**

Related U.S. Application Data

(63) Continuation of application No. 15/913,077, filed on
Mar. 6, 2018, now Pat. No. 11,418,386.

(51) **Int. Cl.**
H04L 41/0806 (2022.01)
G06F 9/455 (2018.01)

(Continued)

(52) **U.S. Cl.**
CPC **H04L 41/0806** (2013.01); **G06F 9/45558**
(2013.01); **G06N 5/04** (2013.01);

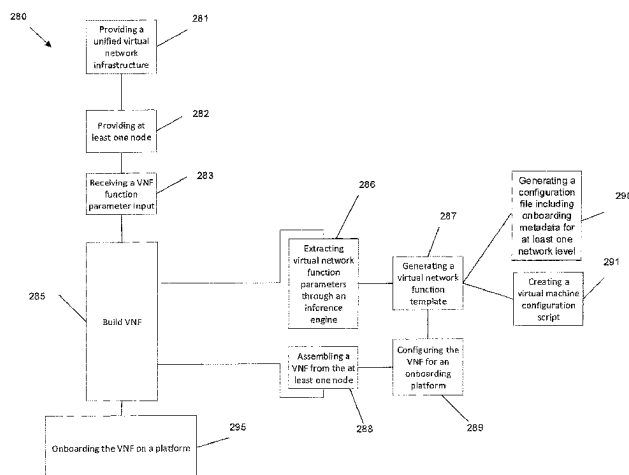
(Continued)

(58) **Field of Classification Search**
None

See application file for complete search history.

A network device comprising: a processor, an input/output
device coupled to the processor, and a memory coupled with
the processor, the memory comprising executable instruc-
tions that when executed by the processor cause the proces-
sor to effectuate operations including instantiating at least
one node comprising a packet processor and a network
interface, the packet processor configured to process a
packet header at a network layer, wherein the at least one
node includes a common configuration; extracting virtual
network function parameters through an inference engine;
generating a virtual network function template based on the
virtual network function parameters, wherein the virtual
network function template instantiates at least one virtual
network function by assembling the at least one virtual
network function from the at least one node; and automati-
cally configures the virtual network function for onboarding
onto a platform.

17 Claims, 13 Drawing Sheets



(51) **Int. Cl.****G06N 5/04** (2023.01)**H04L 41/084** (2022.01)**H04L 69/22** (2022.01)(52) **U.S. Cl.**CPC **H04L 41/0843** (2013.01); **H04L 69/22**
(2013.01); **G06F 2009/45562** (2013.01); **G06F**
2009/45595 (2013.01)(56) **References Cited**

U.S. PATENT DOCUMENTS

10,469,317 B1 * 11/2019 Jiang H04L 41/082
 2015/0326535 A1 * 11/2015 Rao H04L 41/5054
 726/15
 2016/0124781 A1 * 5/2016 Chastain H04L 41/0893
 719/313
 2016/0328259 A1 11/2016 Xia et al.
 2016/0359805 A1 12/2016 Spraggs et al.
 2016/0373474 A1 12/2016 Sood et al.
 2017/0006083 A1 1/2017 McDonnell
 2017/0187572 A1 6/2017 Wu et al.
 2017/0300354 A1 * 10/2017 Dalal H04L 61/5007
 2017/0359231 A1 12/2017 Chawki et al.
 2019/0065232 A1 * 2/2019 Do H04L 41/0806
 2020/0068048 A1 * 2/2020 Hermesh H04L 69/161
 2021/0392056 A1 * 12/2021 Duggal G06Q 30/04

FOREIGN PATENT DOCUMENTS

JP 2015103673 A * 6/2015
 WO 2016107149 A1 7/2016
 WO 2017014803 A1 1/2017
 WO 2017018965 2/2017
 WO 2017035737 A1 3/2017
 WO 2017067567 A1 4/2017
 WO 2017074461 A1 5/2017
 WO 2017107098 A1 6/2017
 WO 2017124801 A1 7/2017
 WO 2017171904 A1 10/2017
 WO 2017178068 A1 10/2017
 WO 2017183565 A1 10/2017
 WO 2017189015 A1 11/2017
 WO 2017197560 A1 11/2017
 WO 2017222595 A3 2/2018

OTHER PUBLICATIONS

Keahey, Katarzyna et al., "Contextualization: Providing One-Click Virtual Clusters", IEEE 4th Int'l Conference, 2008, 8 pages.
 Tutschku, Kurt et al., "On Resource Description Capabilities of On-Board Tools for Resource Management in Cloud Networking and NFV Infrastructures", IEEE Int'l Conf. Communications Workshops, 2016, 6 pages.

* cited by examiner

100

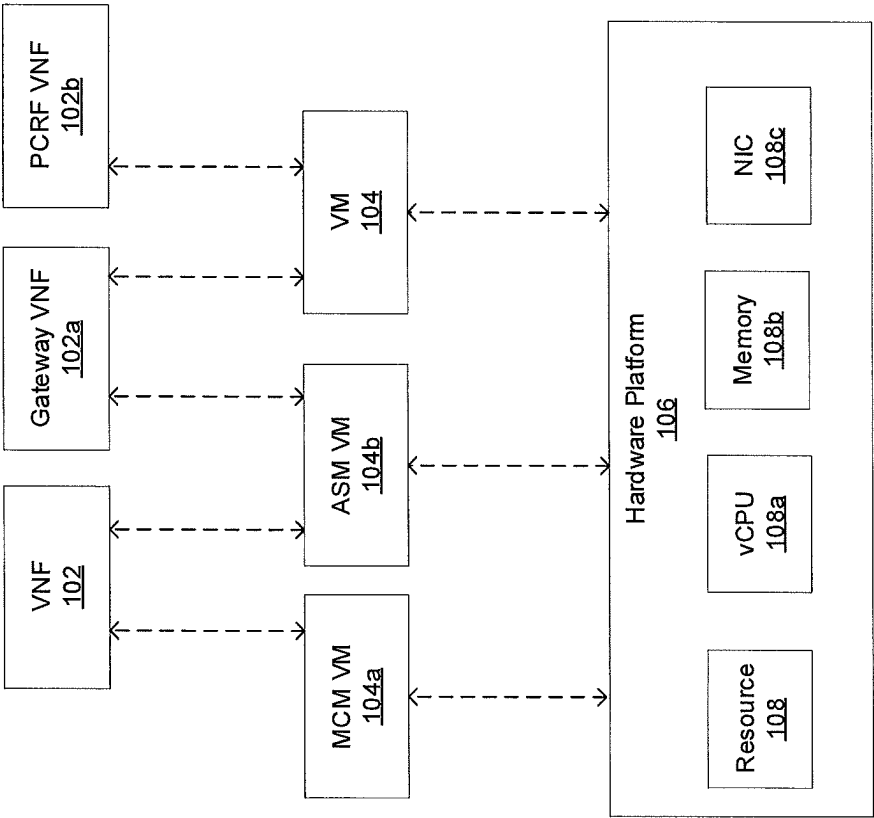


FIG. 1A

106

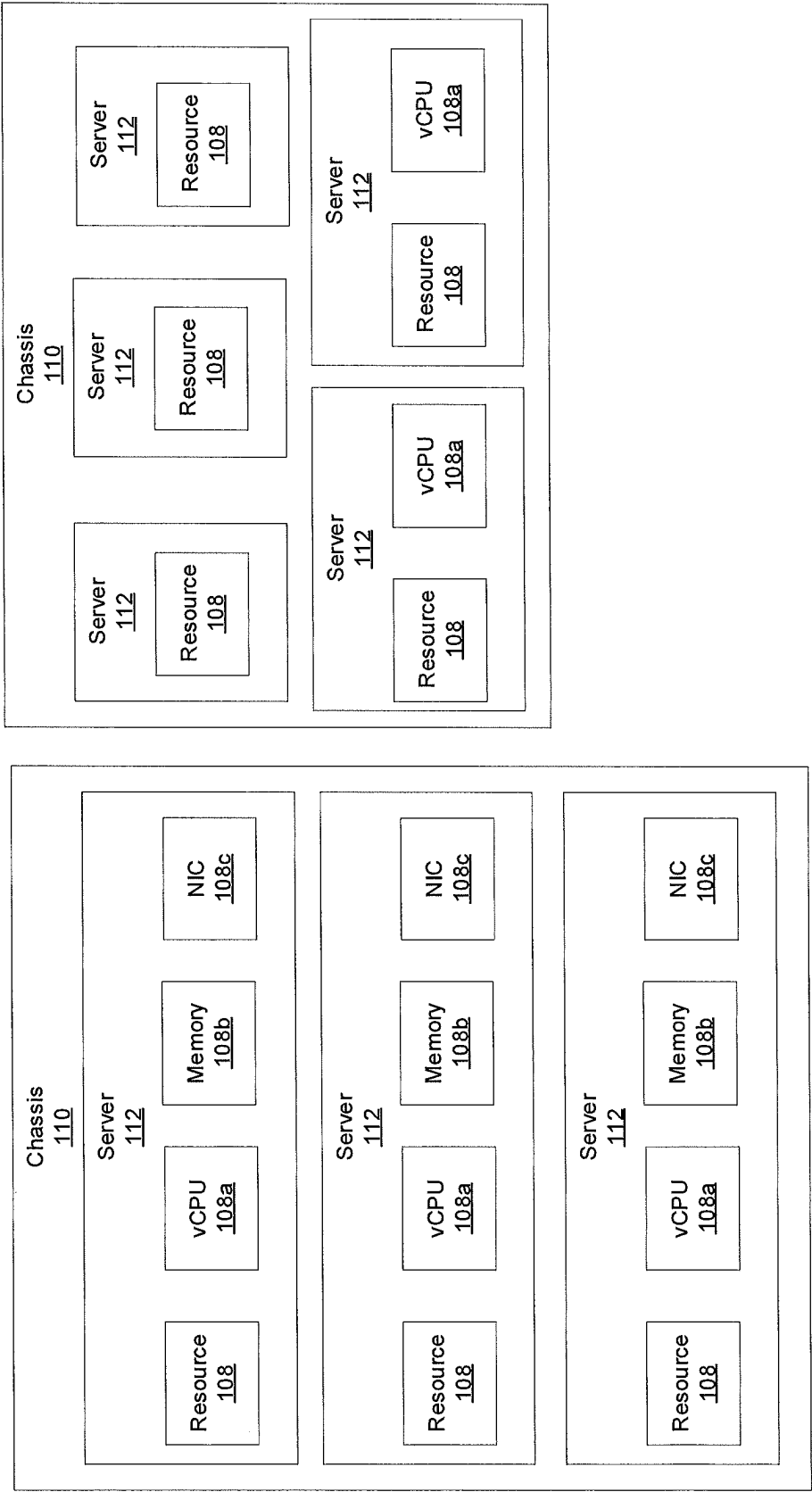


FIG. 1B

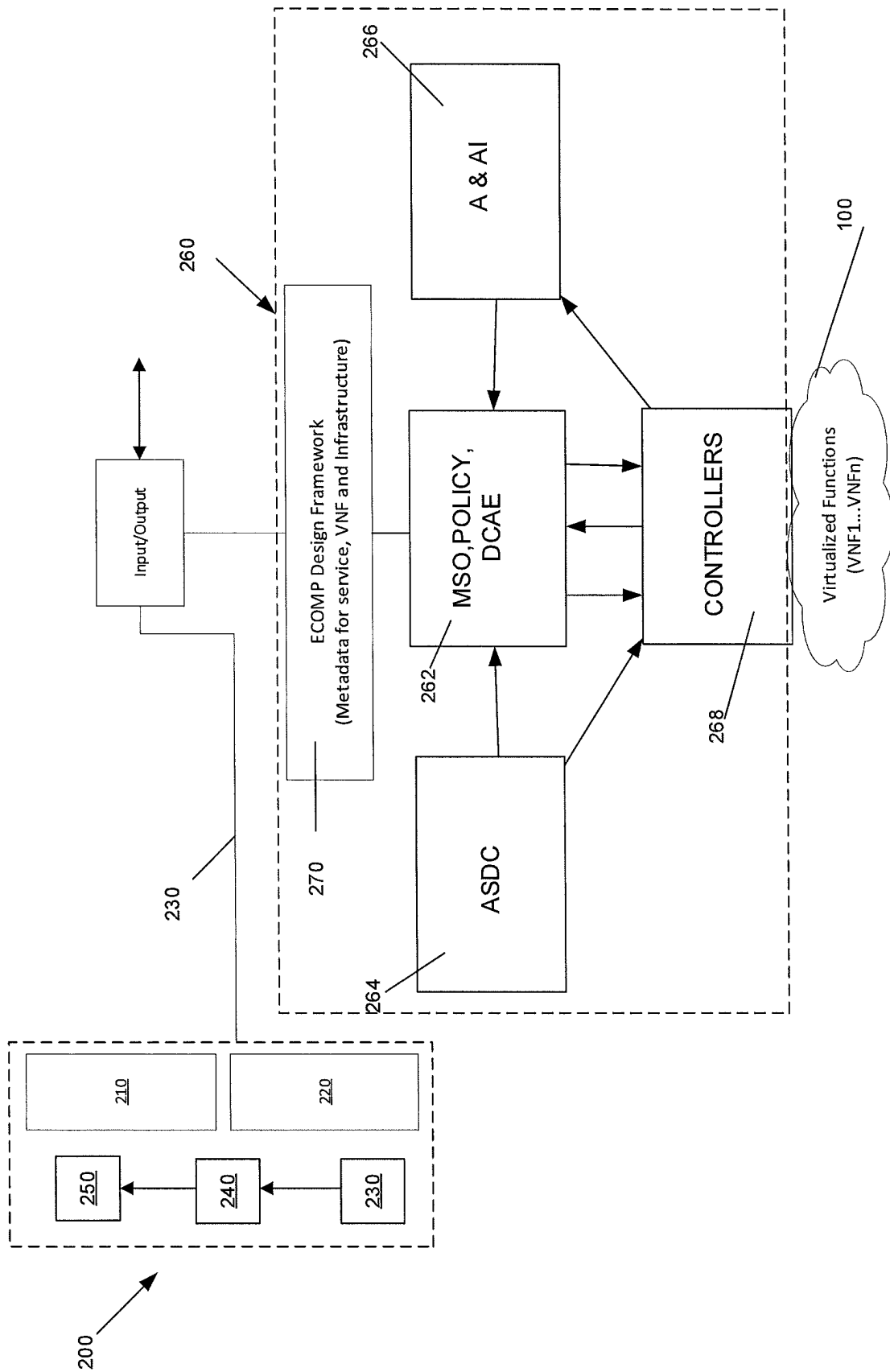


FIG. 2

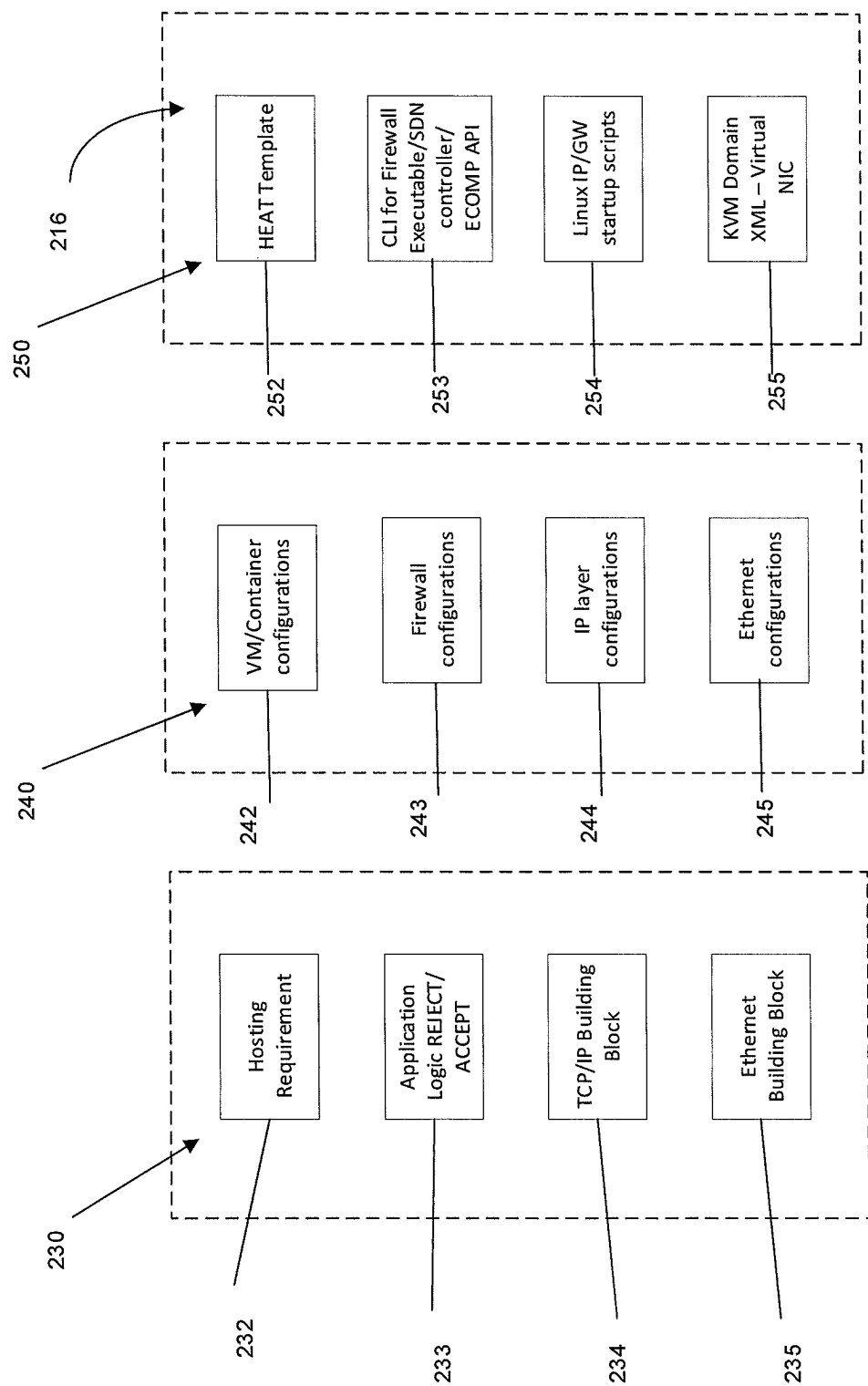
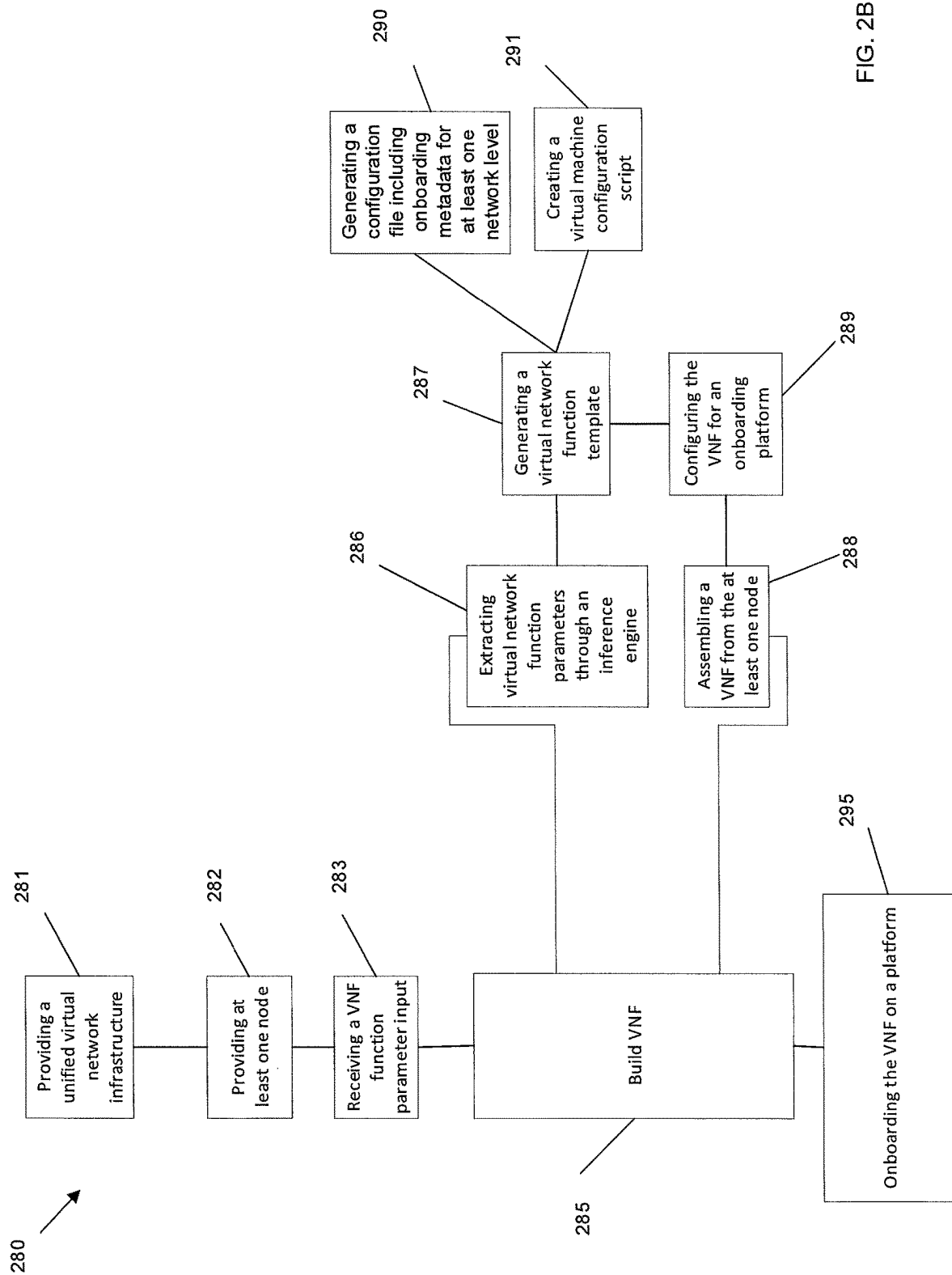


FIG. 2A



300

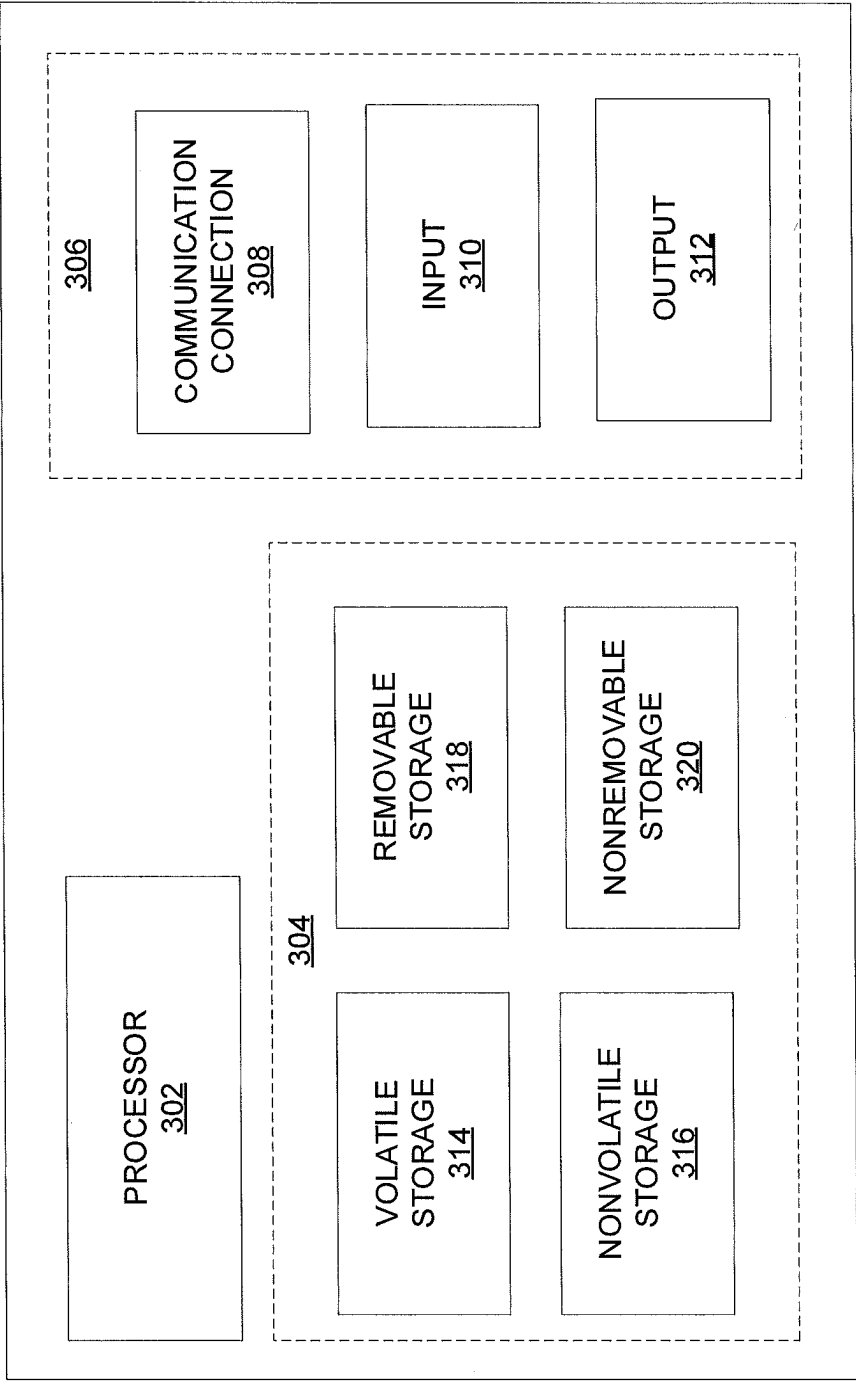


FIG. 3

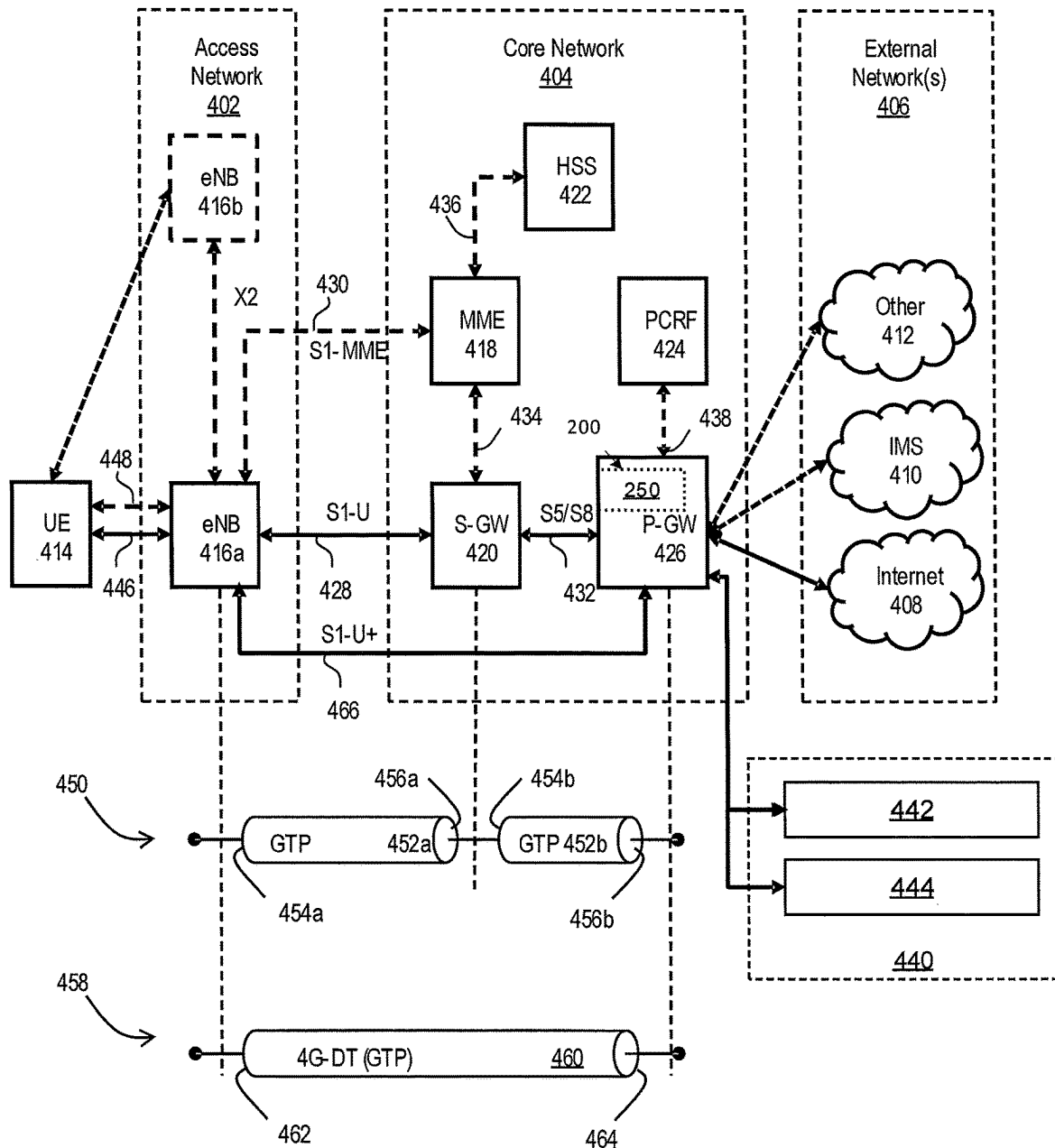
400

FIG. 4

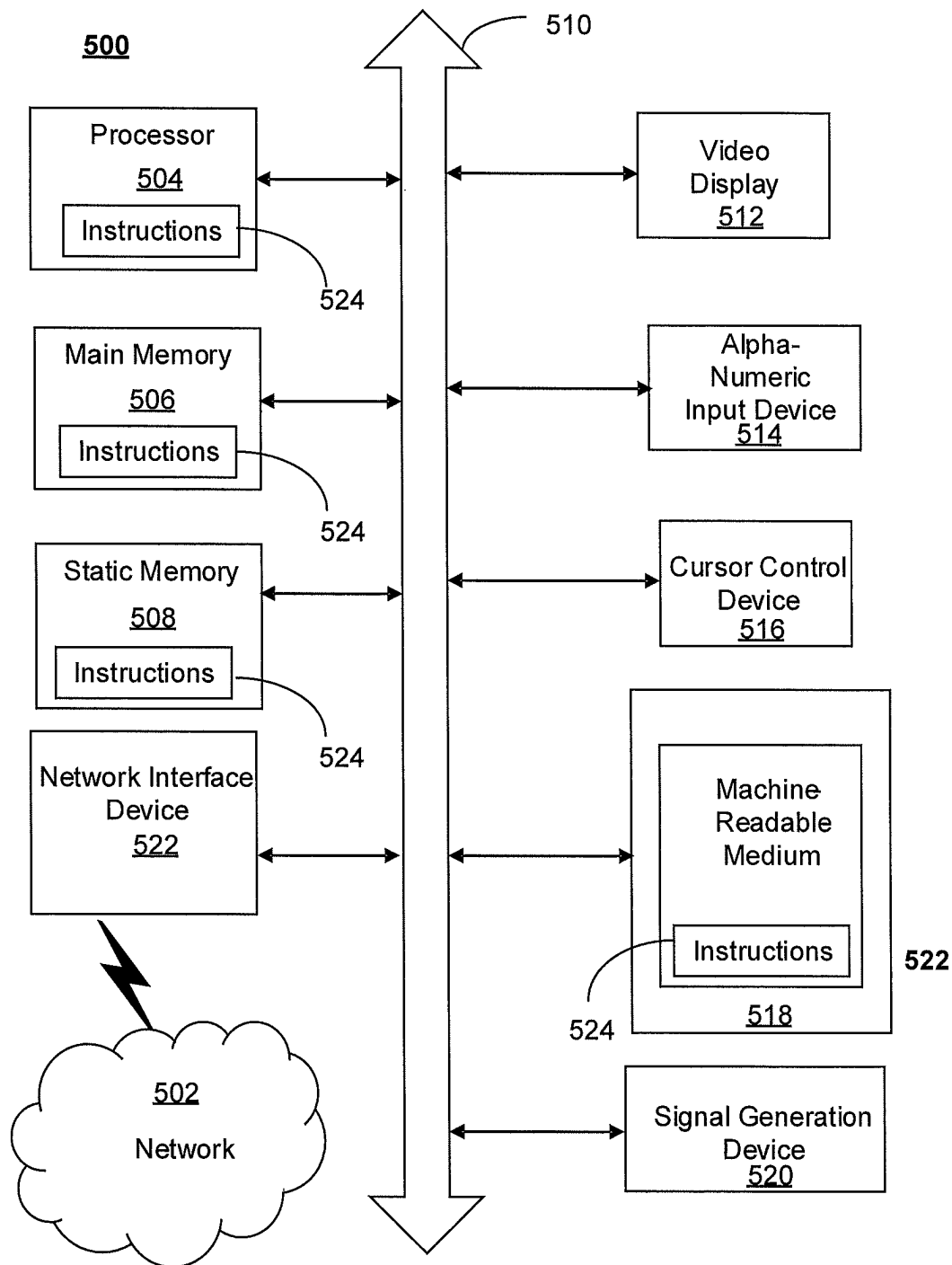


FIG. 5

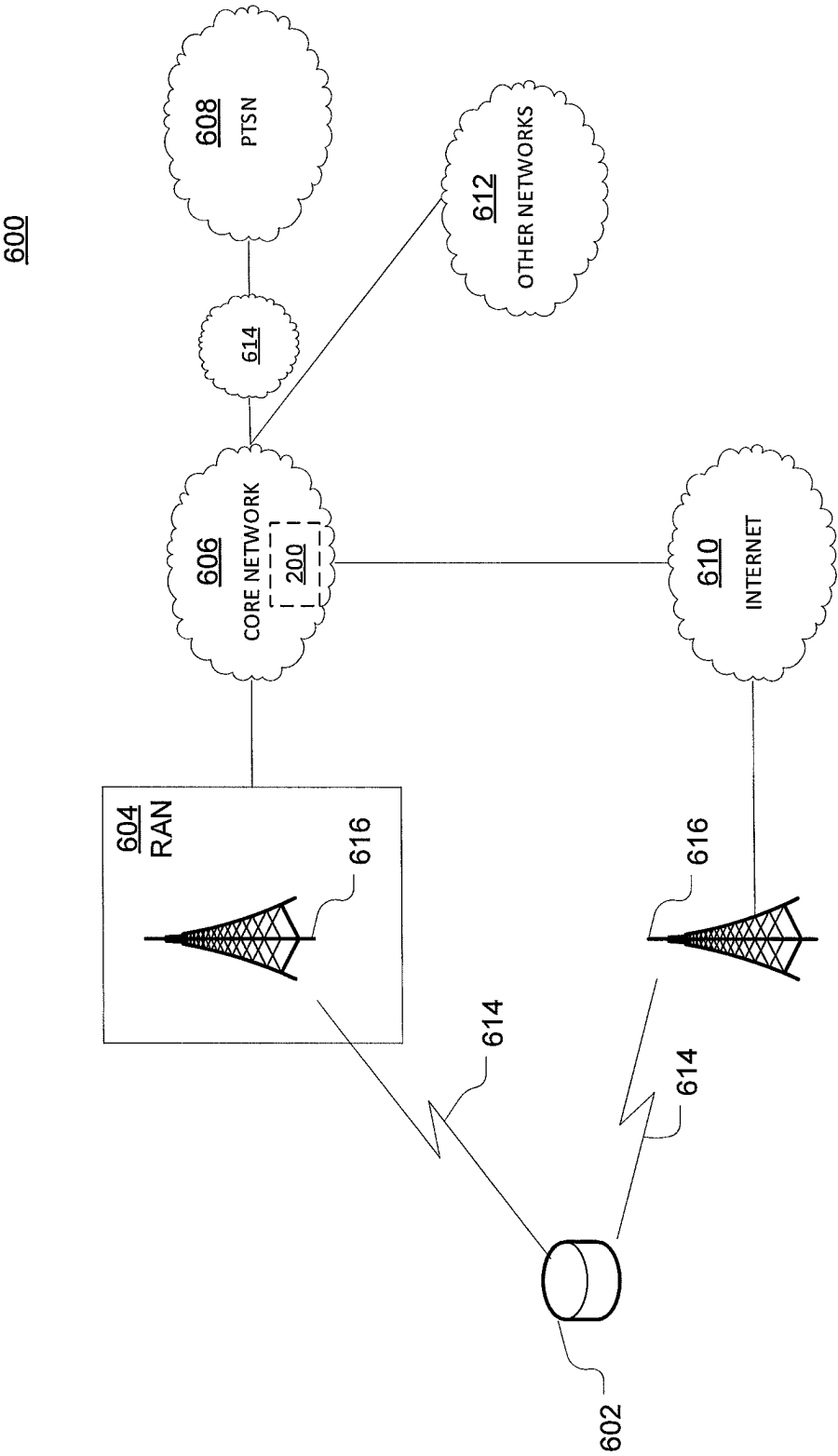


FIG. 6

700

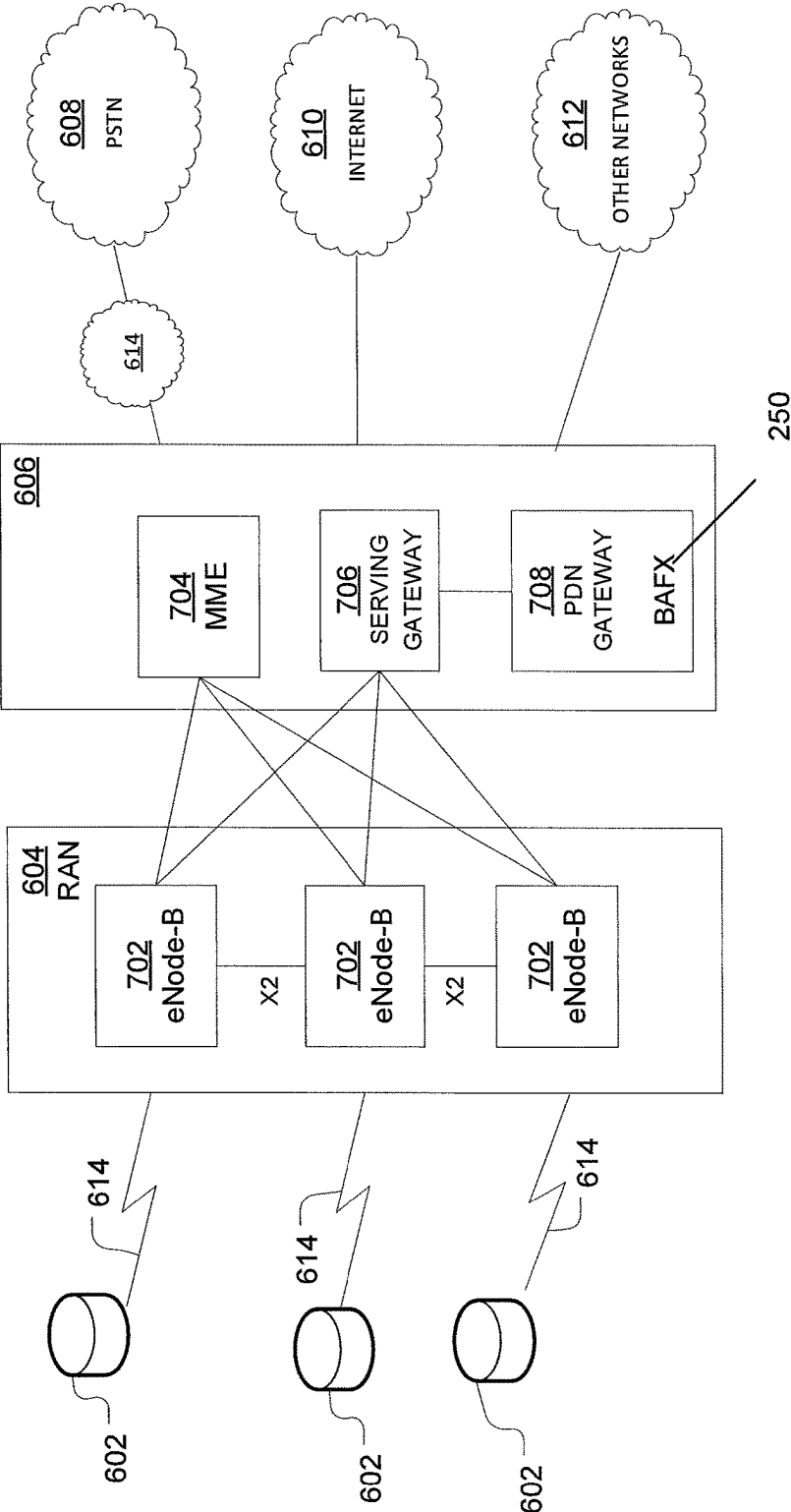


FIG. 7

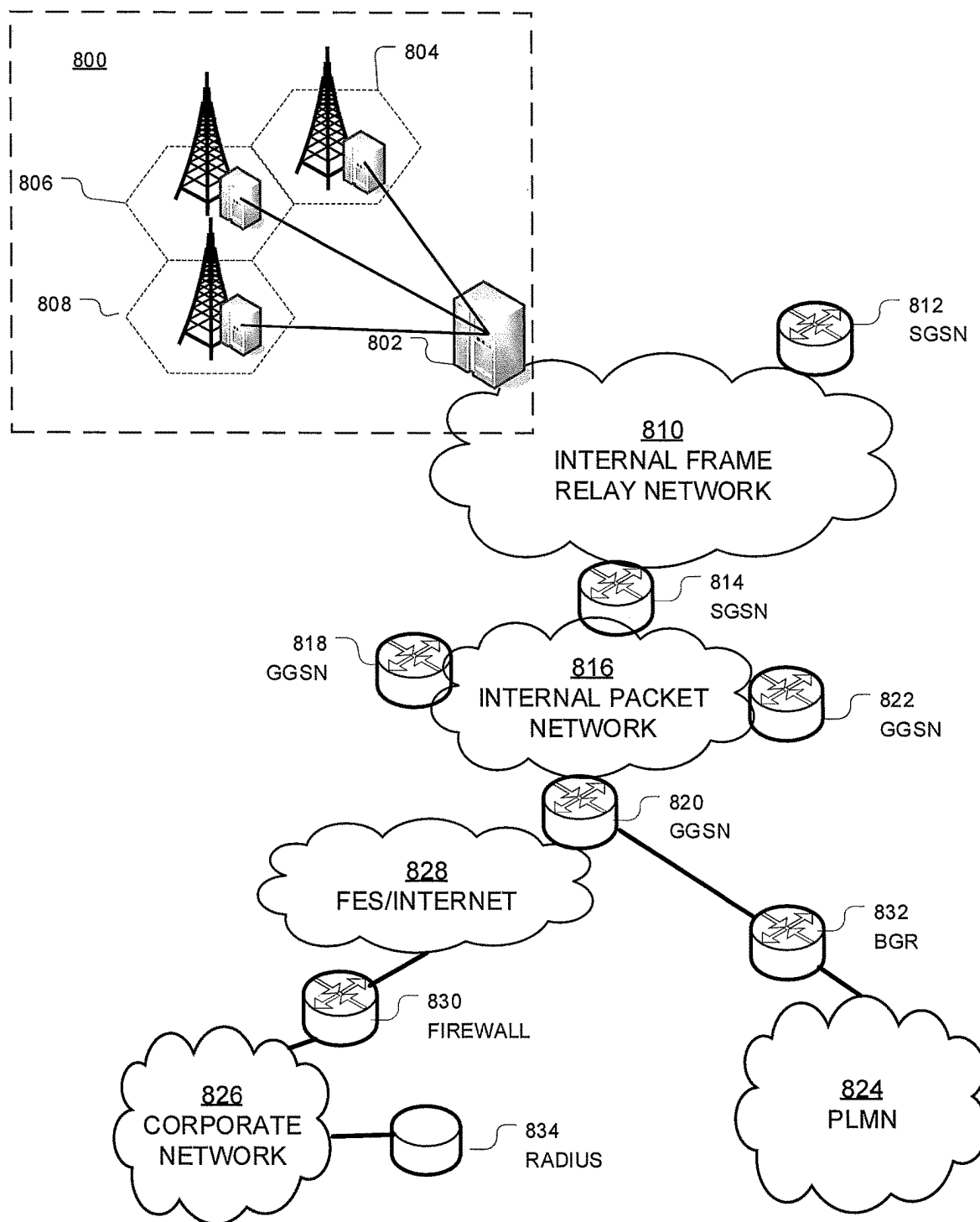


FIG. 8

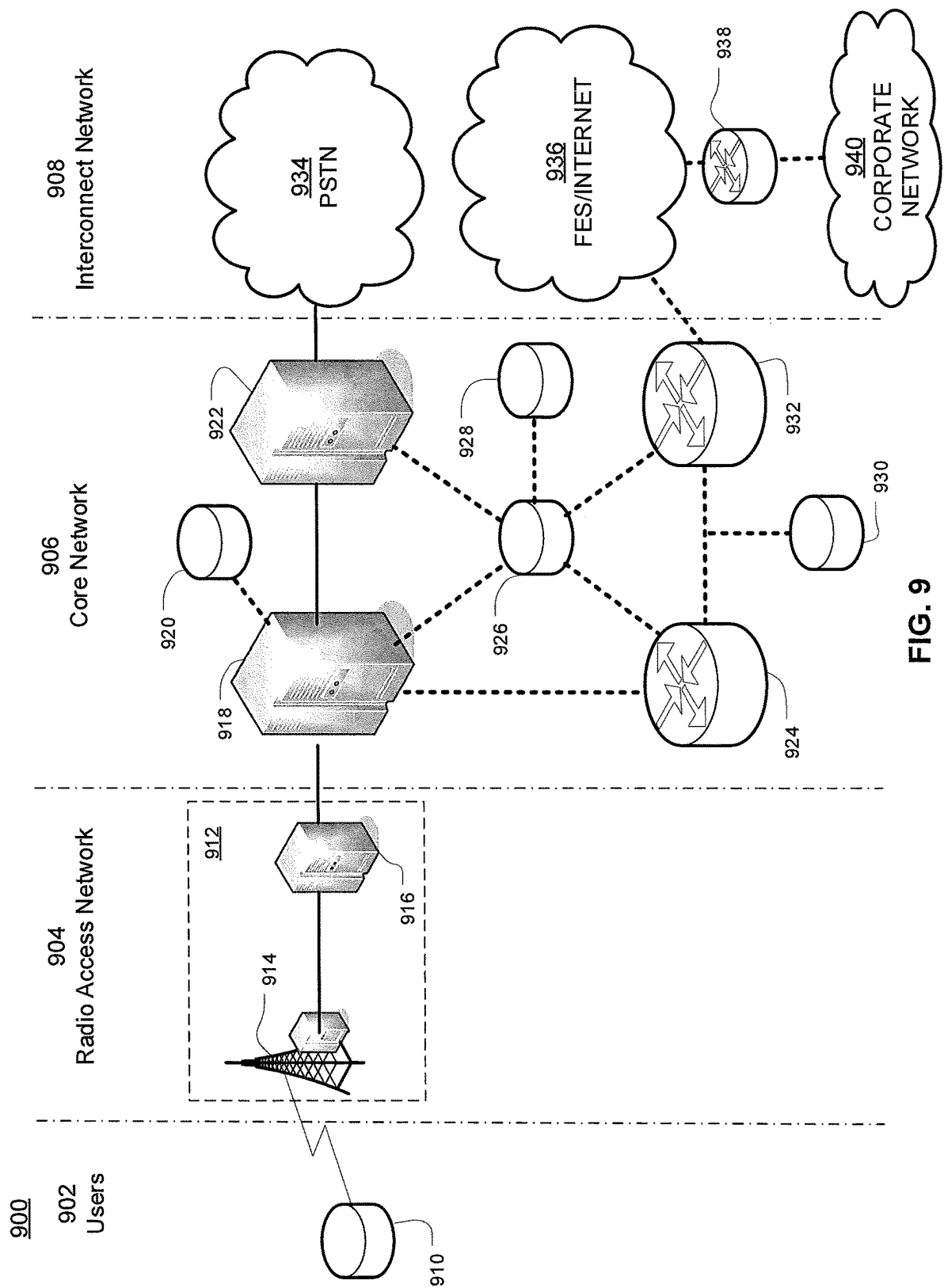


FIG. 9

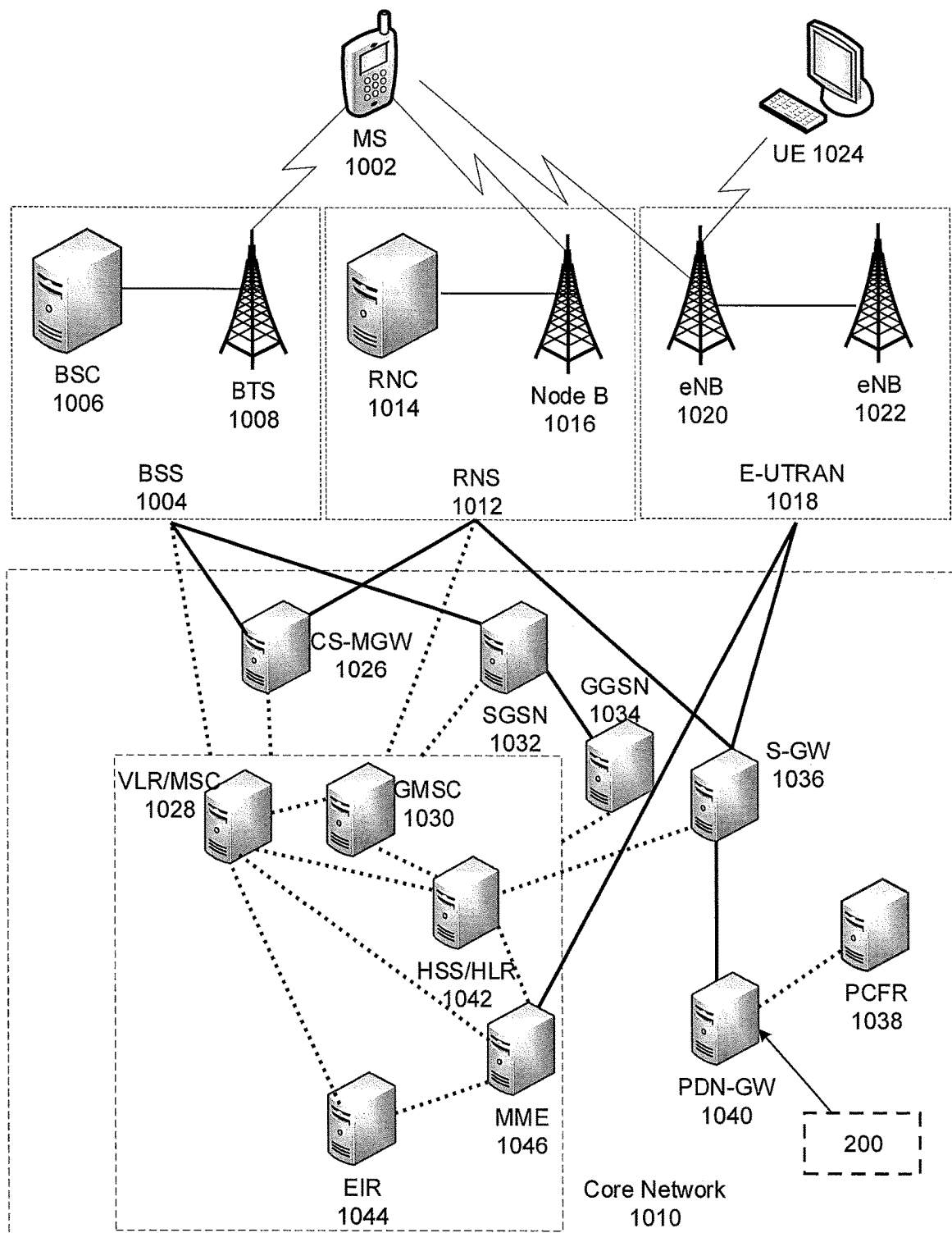


FIG. 10

1

VIRTUAL NETWORK FUNCTION CREATION SYSTEM

CROSS REFERENCE TO RELATED APPLICATIONS

This application is a continuation of U.S. patent application Ser. No. 15/913,077 filed on Mar. 6, 2018. All sections of the aforementioned application are incorporated herein by reference in their entirety.

TECHNICAL FIELD

This disclosure relates generally to network management and, more specifically, to a system for creating virtual network functions.

BACKGROUND

Communication networks have migrated from using specialized networking equipment executing on dedicated hardware, like routers, firewalls, and gateways, to software defined networks (SDNs) executing as virtualized network functions (VNF) in a cloud infrastructure. To provide a service, a set of VNFs may be instantiated on the general purpose hardware. Each VNF may require one or more virtual machines (VMs) to be instantiated. In turn, VMs may require various resources, such as memory, virtual computer processing units (vCPUs), network interfaces or network interface cards (NICs), and servers.

One benefit of cloud computing is the elastic nature of the cloud and the ability to share resources among users and/or spin up virtual resources on demand. Elasticity of resources extends to processing power, storage, bandwidth, virtual functions, and virtual machines. The amount of resources that can be sourced through cloud computing incorporates nearly all facets of computing from raw processing power to storage space. Increasingly, customers are demanding mini-cloud structures to provide localized cloud platforms. Virtual network function (VNF) management sits in the center of network function virtualization and SDN technology. In an SDN based network, the functions need to be efficiently managed in an automated way to minimize service interruptions, maximize performance and reduce human interventions that are typically error prone. As next generation infrastructure is implemented, thousands of VNFs will be deployed increasing the need for automated VNF deployment and management.

Virtual network function creation and onboarding is a process to manage the lifecycle of a VNF in the network. This includes configuring the VNF specifications and other variables according to the application environment; and installing the VNF on physical machines. The process also includes registering the VNF within the software defined network (SDN) platform, and running the VNF code on the physical machines. The process may also include reporting the running status of VNF. Presently, the process includes several manual steps that require user input to create and onboard a VNF. The manual setup of a VNF is error prone. Such errors may prevent proper creation and onboarding of the VNF. Using a firewall VNF as an example, operators need to manually create the virtual function configuration file (called a HEAT file) to specify the number of CPUs, size of memory and disk, etc. required for the VNF. After the VNF is created, operators also need to configure the basic networking parameters such as IP address and gateway. Any mistakes in the onboarding process will cause the VNFs

2

unable to run (correctly). As there are thousands of virtual firewalls in real network environment, the manual configuration does not scale and is time consuming.

To assist in automating the process to deal with issues of scale, one solution is to onboard existing virtual network functions or assemble such functions from pre-existing components or nodes. These solutions must deal with the issue of onboarding pre-existing structures to a different platform. The disclosure includes examples addressing at least one of these problems.

SUMMARY

According to an example, the disclosure generally a network device comprising: a processor, an input/output device coupled to the processor, and a memory coupled with the processor, the memory comprising executable instructions that when executed by the processor cause the processor to effectuate operations including instantiating at least one node comprising a packet processor and a network interface, the packet processor configured to process a packet header at a network layer, wherein the at least one node includes a common configuration; extracting virtual network function parameters through an inference engine; generating a virtual network function template based on the virtual network function parameters, wherein the virtual network function template instantiates at least one virtual network function by assembling the at least one virtual network function from the at least one node; and automatically configures the virtual network function for onboarding onto a platform.

Still another example provides a system for creating a virtual network function comprising: an inference engine in communication with an input device configured to receive a virtual network function input; the inference engine configured to extract at least one virtual network function parameter from the virtual network function input to define a virtual network function template; the virtual network function template comprising at least one of a configuration file and a configuration script, and a packet processing graph; and a builder module, the builder module configured to assemble at least one virtual network function from at least one node according to the virtual network function template and configure the at least one node for onboarding based on the virtual network function template.

Another example provides computer readable storage medium comprising executable instructions that cause a processor to effectuate operations comprising: instantiating at least one node comprising a packet processor and a network interface, the packet processor configured to process a packet header at a network layer, wherein the at least one node includes a common configuration; extracting virtual network function parameters through an inference engine; generating a virtual network function template based on the virtual network function parameters, wherein the virtual network function template instantiates at least one virtual network function by assembling the at least one virtual network function from the at least one node; and automatically configures the virtual network function for onboarding onto a platform.

BRIEF DESCRIPTION OF THE DRAWINGS

In the following description, for purposes of explanation, numerous specific details are set forth in order to provide an understanding of the variations in implementing the disclosed technology. However, the instant disclosure may take

3

many different forms and should not be construed as limited to the examples set forth herein. Where practical, like numbers refer to like elements throughout.

FIG. 1A is a representation of an exemplary network.

FIG. 1B is a representation of an exemplary hardware platform.

FIG. 2 is a representation of an automatic virtual network function creation system according to an example.

FIG. 2A is a representation depicting an inference engine according to an example.

FIG. 2B is a flow diagram depicting operations of a system according to an example.

FIG. 3 is a representation of a network device according to an example.

FIG. 4 depicts an exemplary communication system that provides wireless telecommunication services over wireless communication networks that may be at least partially implemented as an SDN.

FIG. 5 depicts an exemplary diagrammatic representation of a machine in the form of a computer system.

FIG. 6 is a representation of a telecommunications network.

FIG. 7 is a representation of a core network.

FIG. 8 is a representation packet-based mobile cellular network environment.

FIG. 9 is a representation of a GPRS network.

FIG. 10 is a representation a PLMN architecture.

DETAILED DESCRIPTION

FIG. 1A is a representation of an exemplary network 100. Network 100 may comprise a software defined network or SDN—that is, network 100 may include one or more virtualized functions implemented on general purpose hardware, such as in lieu of having dedicated hardware for every network function. General purpose hardware of network 100 may be configured to run virtual network elements to support communication services, such as mobility services, including consumer services and enterprise services. These services may be provided or measured in sessions.

A virtual network function(s) (VNF) 102 may be able to support a limited number of sessions. Each VNF 102 may have a VNF type that indicates its functionality or role. For example, FIG. 1A illustrates a gateway VNF 102a and a policy and charging rules function (PCRF) VNF 102b. Additionally or alternatively, VNFs 102 may include other types of VNFs including but not limited to security, routing, wide area network (WAN) optimization and others within a service providers virtual network offerings.

Each VNF 102 may use one or more virtual machine (VM) 104 to operate. This example, however, is not limiting as VNF may also be implemented as a container based VNF or other similar network namespace. While a VM enables a server to run several guest operating systems and virtualizes hardware using a hypervisor or similar configuration, a container or namespace implementation may remove the hypervisor and run the operating system on the host machine. For example, by adding a container management system (CMS), such as, lightweight LXC or Docker based runtime, applications can run in their own container on top of the operating system to virtualize the operating system. A VNF 102 may thus be supported from a different level within the stack. The VNF 102 is still supported by resources 108 as described in the VM example below.

Each VM 104 may have a VM type that indicates its functionality or role. For example, FIG. 1a illustrates a management control module (MCM) VM 104a and an

4

advanced services module (ASM) VM 104b. Additionally or alternatively, VM 104 may include other types of VMs. Each VM 104 may consume various network resources from a hardware platform 106, such as a resource 108, a virtual central processing unit (vCPU) 108a, memory 108b, or a network interface card (NIC) 108c. Additionally or alternatively, hardware platform 106 may include other types of resources 108.

While FIG. 1A illustrates resources 108 as collectively contained in hardware platform 106, the configuration of hardware platform 106 may isolate, for example, certain memory 108c from other memory 108a. FIG. 1b provides an exemplary implementation of hardware platform 106.

Hardware platform 106 may comprise one or more chassis 110. Chassis 110 may refer to the physical housing or platform for multiple servers or other network equipment. In an aspect, chassis 110 may also refer to the underlying network equipment. Chassis 110 may include one or more servers 112. Server 112 may comprise general purpose computer hardware or a computer. In an aspect, chassis 110 may comprise a metal rack, and servers 112 of chassis 110 may comprise blade servers that are physically mounted in or on chassis 110.

Each server 112 may include one or more network resources 108, as illustrated. Servers 112 may be communicatively coupled together in any combination or arrangement. For example, all servers 112 within a given chassis 110 may be communicatively coupled. As another example, servers 112 in different chassis 110 may be communicatively coupled. Additionally or alternatively, chassis 110 may be communicatively coupled together in any combination or arrangement.

The characteristics of each chassis 110 and each server 112 may differ. For example, FIG. 1B illustrates that the number of servers 112 within two chassis 110 may vary. Additionally or alternatively, the type or number of resources 110 within each server 112 may vary. In an aspect, chassis 110 may be used to group servers 112 with the same resource characteristics. In another aspect, servers 112 within the same chassis 110 may have different resource characteristics.

With reference to FIG. 2, a system for automatically creating at least one VNF is generally indicated by the number 200. System 200 may compose a VNF with reusable network components. Through the process of composing a VNF, system 200 extracts VNF parameters using an inference engine 220 and generates configuration files/scripts to form a virtual network function template, described more completely below. The template may be used to automatically create a VNF and its onboarding programs such that it is ready for deployment. The system 200 then subsequently works with a management platform 260 like ECOMP to finish the onboarding process.

System 200 has common network components or nodes 230 that are building blocks for a VNF. Each node is a packet processing unit with specific logic to process packet header at a network layer. For example, there are Ethernet processing component, IP processing component and TCP/UDP processing component. A VNF can viewed as a composition of nodes to process packet in a specific way. As VNFs have common protocol stacks, the nodes are highly reusable and the creation process is standardized using the same program. Each node has at least one interface to generate configuration files/scripts at each layer. Building a VNF is essentially a way of constructing a graph of nodes and the corresponding onboarding artifacts 250 are generated by invoking the

interfaces/functions associated with each node. As the VNF is created the onboarding files and scripts will be created at the same time.

According to an example, system 200 may be used in connection with a management platform 260, such as an enhanced control, orchestration, management and policy (ECOMP). Platform 260 may include an orchestrator 262, a service and design creator (ASDC) 264, active and available inventory (A&AI) 266 and controllers 268. Functionally, ECOMP includes parallel frameworks including a design framework 270 that builds and sustains models, VNFs, and policies and a runtime framework that applies them to service lifecycle and network infrastructure. ECOMP platform 260 may include a design The system 200 may be implemented in other software defined network platforms and, therefore, discussion of the ECOMP example is not limiting.

System 200 includes a builder 210 that automatically generates metadata and onboarding files to instantiate a VNF. Metadata includes but is not limited to platform templates; local configuration scripts; external configuration scripts; and interfacing with platform components. Platform 260 may have a unified VNF structure, such as for example, a packet processing platform such as VPP. In such a platform, the same code base may be used for all VNFs. This type of platform may also categorize VM and containers. Also, in a platform having a unified structure, the VNFs share a common basic configuration. According to one example, system 200 includes a unified virtual network function infrastructure, such as the ECOMP platform 260. System 200 may instantiate at least one node comprising a packet processor and a network interface, the packet processor configured to process a packet header at a network layer, wherein the at least one node includes a common configuration.

System 200 includes an inference engine, generally indicated by the number 220, that reads functions of a VNF to determine a required configuration. Inference engine 220 identifies a common code structure to extract virtual network parameters and build a virtual network function template 216. For example in a packet processing platform, inference engine 220 may read packet processing logic. The packet processing logic in the example of a VPP may include packet processing graph. Packet processing graph may include at least one of a 1) protocols (IP level information, Mac information, routing table etc.); 2) packet processing details (hardware, software requirements); 3) dependencies; and 4) metrics/logs. Inference engine 220 parses the function information contained in packet processing graph to extract parameters used to create a virtual network function template 216 to allow a VNF to interact with the platform, as described more completely below.

Inference engine 220 may be in communication with building module 210. Building module 210 creates at least one VNF using the virtual network function template 216 from inference engine 220. With reference to FIG. 2A, building module 210 may create artifacts, generally indicated at 240, to instantiate onboarding of the VNF based on a function input, generally indicated by the number 230. Function input 230 is communicated from an input/output device described more completely below and designates a desired function or group of functions for one or more VNFs. Function information 230 may include, but is not limited to a desired function or implementation for at least one VNF. The function and implementation may include the type of function to be performed and the layer at which the VNF will be implemented. In FIG. 2A, the example relates

to a switching function to be implanted within layer 2 of a network. In this example, function information includes at least one of a hosting requirement 232, application logic 233, for example Reject/Accept logic; TCP/IP building blocks 234; and an Ethernet building block 235. Inference engine 220 communicates directly or indirectly with input/output device to receive function information 230.

Based on function information 230, inference engine 220 develops configuration information 216 to create and onboard a VNF. For example, with continued reference to FIG. 2A, inference engine 220 may create artifacts 240 to prepare a VNF for onboarding. In a layer 2 switching example depicted in FIG. 2A, artifacts 240 may include a VM/Container configuration 242, a firewall configuration 243, an IP layer configuration 244, and an Ethernet configuration 245. In addition, inference engine 220 may provide onboarding files or scripts, generally indicated at 250, to allow the builder 220 to create and onboard a VNF automatically. In the example, onboarding files 250 include a HEAT template 252, a CLI for firewall executable/SDN controller/ECOMP API 253, a Linux IP/GW startup script 254, and a KVM domain XML virtual NIC 255. It will be understood that in other VNFs used to provide other functions, different artifacts 240 and onboarding files 250 may be provided by inference engine 220 as part of the virtual network function template 216. Inference engine 220 provides virtual network function template 216 to builder 210. It will be understood that according to an example, builder 210 and inference engine 220 may be incorporated in a single hardware component or network device. In other examples, each may be provided as standalone hardware or instantiated as separate network devices.

With reference to FIG. 2B, system 200 may effectuate operations, generally indicated at 280, including providing a unified virtual network function infrastructure 281; instantiating at least one node 230 at 282. As noted, system 200 may include an input/output device that receives an input specifying a function for a VNF or other parameter input defining a VNF function at 283. Receiving the parameter input 283 prompts the system to perform operations to build a VNF, generally indicated at 285.

The node provided at operation 282 may include a packet processor and a network interface. The packet processor may be configured to process a packet header at a network layer. The node has a common configuration to facilitate its use building a VNF for a variety of functions.

In performing the building operation 285, system 200 extracts virtual network function parameters through an inference engine 220 at 286. Inference engine 220 generates a virtual network function template based on the extracted virtual network function parameters at 287. The virtual network function template instantiates at least one virtual network function by assembling the at least one virtual network function from the at least one node at 288 and automatically configuring the virtual network function for onboarding onto a platform at 289. The step of generating the virtual network function template may include generating a configuration file including onboarding metadata for at least one network level at step 290 and creating a virtual machine configuration script at 291. The VNF built at operation 285 interacts with a platform, such as an ECOMP platform, at operation 295 to onboard the VNF to the platform. By automatically creating both the nodes used to form a VNF and the configuration file, the system 200 and method embodied in its operation more efficiently provides VNFs within a network when compared to systems that attempt to repurpose existing VNFs.

As referenced above, system **200** may be implemented in a network device. FIG. **3** illustrates a functional block diagram depicting one example of a network device, generally indicated at **300**. Network device **300** may comprise a processor **302** and a memory **304** coupled to processor **302**. Memory **304** may contain executable instructions that, when executed by processor **302**, cause processor **302** to effectuate operations associated with building and onboarding at least one VNF as described above. As evident from the description herein, network device **300** is not to be construed as software per se.

In addition to processor **302** and memory **304**, network device **300** may include an input/output system **306**. Processor **302**, memory **304**, and input/output system **306** may be coupled together to allow communications between them. Each portion of network device **300** may comprise circuitry for performing functions associated with each respective portion. Thus, each portion may comprise hardware, or a combination of hardware and software. Accordingly, each portion of network device **300** is not to be construed as software per se. Input/output system **306** may be capable of receiving or providing information from or to a communications device or other network entities configured for telecommunications. For example input/output system **306** may include a wireless communications (e.g., 3G/4G/GPS) card. Input/output system **306** may be capable of receiving or sending video information, audio information, control information, image information, data, or any combination thereof. Input/output system **306** may be capable of transferring information with network device **300**. In various configurations, input/output system **306** may receive or provide information via any appropriate means, such as, for example, optical means (e.g., infrared), electromagnetic means (e.g., RF, Wi-Fi, Bluetooth®, ZigBee®), acoustic means (e.g., speaker, microphone, ultrasonic receiver, ultrasonic transmitter), electrical means, or a combination thereof. In an example configuration, input/output system **306** may comprise a Wi-Fi finder, a two-way GPS chipset or equivalent, or the like, or a combination thereof. Bluetooth, infrared, NFC, and Zigbee are generally considered short range (e.g., few centimeters to 20 meters). WiFi is considered medium range (e.g., approximately 100 meters).

Input/output system **306** of network device **300** also may contain a communication connection **308** that allows network device **300** to communicate with other devices, network entities, or the like. Communication connection **308** may comprise communication media. Communication media typically embody computer-readable instructions, data structures, program modules or other data in a modulated data signal such as a carrier wave or other transport mechanism and includes any information delivery media. By way of example, and not limitation, communication media may include wired media such as a wired network or direct-wired connection, or wireless media such as acoustic, RF, infrared, or other wireless media. The term computer-readable media as used herein includes both storage media and communication media. Input/output system **306** also may include an input device **310** such as keyboard, mouse, pen, voice input device, or touch input device. Input/output system **306** may also include an output device **312**, such as a display, speakers, or a printer.

Processor **302** may be capable of performing functions associated with telecommunications, such as functions for processing broadcast messages, as described herein. For example, processor **302** may be capable of, in conjunction with any other portion of network device **300**, determining

a type of broadcast message and acting according to the broadcast message type or content, as described herein.

Memory **304** of network device **300** may comprise a storage medium having a concrete, tangible, physical structure. As is known, a signal does not have a concrete, tangible, physical structure. Memory **304**, as well as any computer-readable storage medium described herein, is not to be construed as a signal. Memory **304**, as well as any computer-readable storage medium described herein, is not to be construed as a transient signal. Memory **304**, as well as any computer-readable storage medium described herein, is not to be construed as a propagating signal. Memory **304**, as well as any computer-readable storage medium described herein, is to be construed as an article of manufacture.

Memory **304** may store any information utilized in conjunction with telecommunications. Depending upon the exact configuration or type of processor, memory **304** may include a volatile storage **314** (such as some types of RAM), a nonvolatile storage **316** (such as ROM, flash memory), or a combination thereof. Memory **304** may include additional storage (e.g., a removable storage **318** or a non-removable storage **320**) including, for example, tape, flash memory, smart cards, CD-ROM, DVD, or other optical storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, USB-compatible memory, or any other medium that can be used to store information and that can be accessed by network device **300**. Memory **304** may comprise executable instructions that, when executed by processor **302**, cause processor **302** to effectuate operations to create at least one VNF as described above.

As discussed previously, a system **200** including inference engine **220** and builder **210** may be incorporated in an SDN **100**. SDN **100** may be implemented in a variety of network architectures. Example architectures will now be described with reference to FIGS. **4-10**. These examples should not be considered limiting. FIG. **4** illustrates a functional block diagram depicting one example of an LTE-EPS network architecture **400** that may be at least partially implemented as an SDN **100**. Network architecture **400** disclosed herein is referred to as a modified LTE-EPS architecture **400** to distinguish it from a traditional LTE-EPS architecture.

An example modified LTE-EPS architecture **400** is based at least in part on standards developed by the 3rd Generation Partnership Project (3GPP), with information available at www.3gpp.org. LTE-EPS network architecture **400** may include an access network **402**, a core network **404**, e.g., an EPC or Common BackBone (CBB) and one or more external networks **406**, sometimes referred to as PDN or peer entities. Different external networks **406** can be distinguished from each other by a respective network identifier, e.g., a label according to DNS naming conventions describing an access point to the PDN. Such labels can be referred to as Access Point Names (APN). External networks **406** can include one or more trusted and non-trusted external networks such as an internet protocol (IP) network **408**, an IP multimedia subsystem (IMS) network **410**, and other networks **412**, such as a service network, a corporate network, or the like. In an aspect, access network **402**, core network **404**, or external network **405** may include or communicate with network **100**.

Access network **402** can include an LTE network architecture sometimes referred to as Evolved Universal mobile Telecommunication system Terrestrial Radio Access (E-UTRA) and evolved UMTS Terrestrial Radio Access Network (E-UTRAN). Broadly, access network **402** can include one or more communication devices, commonly referred to as UE **414**, and one or more wireless access nodes, or base

stations **416a**, **416b**. During network operations, at least one base station **416** communicates directly with UE **414**. Base station **416** can be an evolved Node B (e-NodeB), with which UE **414** communicates over the air and wirelessly. UEs **414** can include, without limitation, wireless devices, e.g., satellite communication systems, portable digital assistants (PDAs), laptop computers, tablet devices and other mobile devices (e.g., cellular telephones, smart appliances, and so on). UEs **414** can connect to eNBs **416** when UE **414** is within range according to a corresponding wireless communication technology.

UE **414** generally runs one or more applications that engage in a transfer of packets between UE **414** and one or more external networks **406**. Such packet transfers can include one of downlink packet transfers from external network **406** to UE **414**, uplink packet transfers from UE **414** to external network **406** or combinations of uplink and downlink packet transfers. Applications can include, without limitation, web browsing, VoIP, streaming media and the like. Each application can pose different Quality of Service (QoS) requirements on a respective packet transfer. Different packet transfers can be served by different bearers within core network **404**, e.g., according to parameters, such as the QoS.

Core network **404** uses a concept of bearers, e.g., EPS bearers, to route packets, e.g., IP traffic, between a particular gateway in core network **404** and UE **414**. A bearer refers generally to an IP packet flow with a defined QoS between the particular gateway and UE **414**. Access network **402**, e.g., E UTRAN, and core network **404** together set up and release bearers as required by the various applications. Bearers can be classified in at least two different categories: (i) minimum guaranteed bit rate bearers, e.g., for applications, such as VoIP; and (ii) non-guaranteed bit rate bearers that do not require guarantee bit rate, e.g., for applications, such as web browsing.

In one embodiment, the core network **404** includes various network entities, such as MME **418**, SGW **420**, Home Subscriber Server (HSS) **422**, Policy and Charging Rules Function (PCRF) **424** and PGW **426**. In one embodiment, MME **418** comprises a control node performing a control signaling between various equipment and devices in access network **402** and core network **404**. The protocols running between UE **414** and core network **404** are generally known as Non-Access Stratum (NAS) protocols.

For illustration purposes only, the terms MME **418**, SGW **420**, HSS **422** and PGW **426**, and so on, can be server devices, but may be referred to in the subject disclosure without the word "server." It is also understood that any form of such servers can operate in a device, system, component, or other form of centralized or distributed hardware and software. It is further noted that these terms and other terms such as bearer paths and/or interfaces are terms that can include features, methodologies, and/or fields that may be described in whole or in part by standards bodies such as the 3GPP. It is further noted that some or all embodiments of the subject disclosure may in whole or in part modify, supplement, or otherwise supersede final or proposed standards published and promulgated by 3GPP.

According to traditional implementations of LTE-EPS architectures, SGW **420** routes and forwards all user data packets. SGW **420** also acts as a mobility anchor for user plane operation during handovers between base stations, e.g., during a handover from first eNB **416a** to second eNB **416b** as may be the result of UE **414** moving from one area of coverage, e.g., cell, to another. SGW **420** can also terminate a downlink data path, e.g., from external network

406 to UE **414** in an idle state, and trigger a paging operation when downlink data arrives for UE **414**. SGW **420** can also be configured to manage and store a context for UE **414**, e.g., including one or more of parameters of the IP bearer service and network internal routing information. In addition, SGW **420** can perform administrative functions, e.g., in a visited network, such as collecting information for charging (e.g., the volume of data sent to or received from the user), and/or replicate user traffic, e.g., to support a lawful interception. SGW **420** also serves as the mobility anchor for interworking with other 3GPP technologies such as universal mobile telecommunication system (UMTS).

At any given time, UE **414** is generally in one of three different states: detached, idle, or active. The detached state is typically a transitory state in which UE **414** is powered on but is engaged in a process of searching and registering with network **402**. In the active state, UE **414** is registered with access network **402** and has established a wireless connection, e.g., radio resource control (RRC) connection, with eNB **416**. Whether UE **414** is in an active state can depend on the state of a packet data session, and whether there is an active packet data session. In the idle state, UE **414** is generally in a power conservation state in which UE **414** typically does not communicate packets. When UE **414** is idle, SGW **420** can terminate a downlink data path, e.g., from one peer entity **406**, and triggers paging of UE **414** when data arrives for UE **414**. If UE **414** responds to the page, SGW **420** can forward the IP packet to eNB **416a**.

HSS **422** can manage subscription-related information for a user of UE **414**. For example, HSS **422** can store information such as authorization of the user, security requirements for the user, quality of service (QoS) requirements for the user, etc. HSS **422** can also hold information about external networks **406** to which the user can connect, e.g., in the form of an APN of external networks **406**. For example, MME **418** can communicate with HSS **422** to determine if UE **414** is authorized to establish a call, e.g., a voice over IP (VoIP) call before the call is established.

PCRF **424** can perform QoS management functions and policy control. PCRF **424** is responsible for policy control decision-making, as well as for controlling the flow-based charging functionalities in a policy control enforcement function (PCEF), which resides in PGW **426**. PCRF **424** provides the QoS authorization, e.g., QoS class identifier and bit rates that decide how a certain data flow will be treated in the PCEF and ensures that this is in accordance with the user's subscription profile.

PGW **426** can provide connectivity between the UE **414** and one or more of the external networks **406**. In illustrative network architecture **400**, PGW **426** can be responsible for IP address allocation for UE **414**, as well as one or more of QoS enforcement and flow-based charging, e.g., according to rules from the PCRF **424**. PGW **426** is also typically responsible for filtering downlink user IP packets into the different QoS-based bearers. In at least some embodiments, such filtering can be performed based on traffic flow templates. PGW **426** can also perform QoS enforcement, e.g., for guaranteed bit rate bearers. PGW **426** also serves as a mobility anchor for interworking with non-3GPP technologies such as CDMA2000.

Within access network **402** and core network **404** there may be various bearer paths/interfaces, e.g., represented by solid lines **428** and **430**. Some of the bearer paths can be referred to by a specific label. For example, solid line **428** can be considered an S1-U bearer and solid line **432** can be considered an S5/S8 bearer according to LTE-EPS architecture standards. Without limitation, reference to various inter-

11

faces, such as S1, X2, S5, S8, S11 refer to EPS interfaces. In some instances, such interface designations are combined with a suffix, e.g., a “U” or a “C” to signify whether the interface relates to a “User plane” or a “Control plane.” In addition, the core network **404** can include various signaling bearer paths/interfaces, e.g., control plane paths/interfaces represented by dashed lines **430**, **434**, **436**, and **438**. Some of the signaling bearer paths may be referred to by a specific label. For example, dashed line **430** can be considered as an S1-MME signaling bearer, dashed line **434** can be considered as an S11 signaling bearer and dashed line **436** can be considered as an S6a signaling bearer, e.g., according to LTE-EPS architecture standards. The above bearer paths and signaling bearer paths are only illustrated as examples and it should be noted that additional bearer paths and signaling bearer paths may exist that are not illustrated.

Also shown is a novel user plane path/interface, referred to as the S1-U+ interface **466**. In the illustrative example, the S1-U+ user plane interface extends between the eNB **416a** and PGW **426**. Notably, S1-U+ path/interface does not include SGW **420**, a node that is otherwise instrumental in configuring and/or managing packet forwarding between eNB **416a** and one or more external networks **406** by way of PGW **426**. As disclosed herein, the S1-U+ path/interface facilitates autonomous learning of peer transport layer addresses by one or more of the network nodes to facilitate a self-configuring of the packet forwarding path. In particular, such self-configuring can be accomplished during handovers in most scenarios so as to reduce any extra signaling load on the S/PGWs **420**, **426** due to excessive handover events.

In some embodiments, PGW **426** is coupled to storage device **440**, shown in phantom. Storage device **440** can be integral to one of the network nodes, such as PGW **426**, for example, in the form of internal memory and/or disk drive. It is understood that storage device **440** can include registers suitable for storing address values. Alternatively or in addition, storage device **440** can be separate from PGW **426**, for example, as an external hard drive, a flash drive, and/or network storage.

Storage device **440** selectively stores one or more values relevant to the forwarding of packet data. For example, storage device **440** can store identities and/or addresses of network entities, such as any of network nodes **418**, **420**, **422**, **424**, and **426**, eNBs **416** and/or UE **414**. In the illustrative example, storage device **440** includes a first storage location **442** and a second storage location **444**. First storage location **442** can be dedicated to storing a Currently Used Downlink address value **442**. Likewise, second storage location **444** can be dedicated to storing a Default Downlink Forwarding address value **444**. PGW **426** can read and/or write values into either of storage locations **442**, **444**, for example, managing Currently Used Downlink Forwarding address value **442** and Default Downlink Forwarding address value **444** as disclosed herein.

In some embodiments, the Default Downlink Forwarding address for each EPS bearer is the SGW S5-U address for each EPS Bearer. The Currently Used Downlink Forwarding address” for each EPS bearer in PGW **426** can be set every time when PGW **426** receives an uplink packet, e.g., a GTP-U uplink packet, with a new source address for a corresponding EPS bearer. When UE **414** is in an idle state, the “Current Used Downlink Forwarding address” field for each EPS bearer of UE **414** can be set to a “null” or other suitable value.

In some embodiments, the Default Downlink Forwarding address is only updated when PGW **426** receives a new

12

SGW S5-U address in a predetermined message or messages. For example, the Default Downlink Forwarding address is only updated when PGW **426** receives one of a Create Session Request, Modify Bearer Request and Create Bearer Response messages from SGW **420**.

As values **442**, **444** can be maintained and otherwise manipulated on a per bearer basis, it is understood that the storage locations can take the form of tables, spreadsheets, lists, and/or other data structures generally well understood and suitable for maintaining and/or otherwise manipulate forwarding addresses on a per bearer basis.

It should be noted that access network **402** and core network **404** are illustrated in a simplified block diagram in FIG. **4**. In other words, either or both of access network **402** and the core network **404** can include additional network elements that are not shown, such as various routers, switches and controllers. In addition, although FIG. **4** illustrates only a single one of each of the various network elements, it should be noted that access network **402** and core network **404** can include any number of the various network elements. For example, core network **404** can include a pool (i.e., more than one) of MMEs **418**, SGWs **420** or PGWs **426**.

In the illustrative example, data traversing a network path between UE **414**, eNB **416a**, SGW **420**, PGW **426** and external network **406** may be considered to constitute data transferred according to an end-to-end IP service. However, for the present disclosure, to properly perform establishment management in LTE-EPS network architecture **400**, the core network, data bearer portion of the end-to-end IP service is analyzed.

An establishment may be defined herein as a connection set up request between any two elements within LTE-EPS network architecture **400**. The connection set up request may be for user data or for signaling. A failed establishment may be defined as a connection set up request that was unsuccessful. A successful establishment may be defined as a connection set up request that was successful.

In one embodiment, a data bearer portion comprises a first portion (e.g., a data radio bearer **446**) between UE **414** and eNB **416a**, a second portion (e.g., an S1 data bearer **428**) between eNB **416a** and SGW **420**, and a third portion (e.g., an S5/S8 bearer **432**) between SGW **420** and PGW **426**. Various signaling bearer portions are also illustrated in FIG. **4**. For example, a first signaling portion (e.g., a signaling radio bearer **448**) between UE **414** and eNB **416a**, and a second signaling portion (e.g., S1 signaling bearer **430**) between eNB **416a** and MME **418**.

In at least some embodiments, the data bearer can include tunneling, e.g., IP tunneling, by which data packets can be forwarded in an encapsulated manner, between tunnel endpoints. Tunnels, or tunnel connections can be identified in one or more nodes of network **100**, e.g., by one or more of tunnel endpoint identifiers, an IP address and a user data-gram protocol port number. Within a particular tunnel connection, payloads, e.g., packet data, which may or may not include protocol related information, are forwarded between tunnel endpoints.

An example of first tunnel solution **450** includes a first tunnel **452a** between two tunnel endpoints **454a** and **456a**, and a second tunnel **452b** between two tunnel endpoints **454b** and **456b**. In the illustrative example, first tunnel **452a** is established between eNB **416a** and SGW **420**. Accordingly, first tunnel **452a** includes a first tunnel endpoint **454a** corresponding to an S1-U address of eNB **416a** (referred to herein as the eNB S1-U address), and second tunnel endpoint **456a** corresponding to an S1-U address of SGW **420**.

13

(referred to herein as the SGW S1-U address). Likewise, second tunnel **452b** includes first tunnel endpoint **454b** corresponding to an S5-U address of SGW **420** (referred to herein as the SGW S5-U address), and second tunnel endpoint **456b** corresponding to an S5-U address of PGW **426** (referred to herein as the PGW S5-U address).

In at least some embodiments, first tunnel solution **450** is referred to as a two tunnel solution, e.g., according to the GPRS Tunneling Protocol User Plane (GTPv1-U based), as described in 3GPP specification TS 29.281, incorporated herein in its entirety. It is understood that one or more tunnels are permitted between each set of tunnel end points. For example, each subscriber can have one or more tunnels, e.g., one for each PDP context that they have active, as well as possibly having separate tunnels for specific connections with different quality of service requirements, and so on.

An example of second tunnel solution **458** includes a single or direct tunnel **460** between tunnel endpoints **462** and **464**. In the illustrative example, direct tunnel **460** is established between eNB **416a** and PGW **426**, without subjecting packet transfers to processing related to SGW **420**. Accordingly, direct tunnel **460** includes first tunnel endpoint **462** corresponding to the eNB S1-U address, and second tunnel endpoint **464** corresponding to the PGW S5-U address. Packet data received at either end can be encapsulated into a payload and directed to the corresponding address of the other end of the tunnel. Such direct tunneling avoids processing, e.g., by SGW **420** that would otherwise relay packets between the same two endpoints, e.g., according to a protocol, such as the GTP-U protocol.

In some scenarios, direct tunneling solution **458** can forward user plane data packets between eNB **416a** and PGW **426**, by way of SGW **420**. That is, SGW **420** can serve a relay function, by relaying packets between two tunnel endpoints **416a**, **426**. In other scenarios, direct tunneling solution **458** can forward user data packets between eNB **416a** and PGW **426**, by way of the S1-U+ interface, thereby bypassing SGW **420**.

Generally, UE **414** can have one or more bearers at any one time. The number and types of bearers can depend on applications, default requirements, and so on. It is understood that the techniques disclosed herein, including the configuration, management and use of various tunnel solutions **450**, **458**, can be applied to the bearers on an individual bases. That is, if user data packets of one bearer, say a bearer associated with a VoIP service of UE **414**, then the forwarding of all packets of that bearer are handled in a similar manner. Continuing with this example, the same UE **414** can have another bearer associated with it through the same eNB **416a**. This other bearer, for example, can be associated with a relatively low rate data session forwarding user data packets through core network **404** simultaneously with the first bearer. Likewise, the user data packets of the other bearer are also handled in a similar manner, without necessarily following a forwarding path or solution of the first bearer. Thus, one of the bearers may be forwarded through direct tunnel **458**; whereas, another one of the bearers may be forwarded through a two-tunnel solution **450**.

FIG. 5 depicts an exemplary diagrammatic representation of a machine in the form of a computer system **500** within which a set of instructions, when executed, may cause the machine to perform any one or more of the methods described above. One or more instances of the machine can operate, for example, as processor **302**, UE **414**, eNB **416**, MME **418**, SGW **420**, HSS **422**, PCRF **424**, PGW **426** and other devices of FIGS. 1, 2, and 4. In some embodiments, the machine may be connected (e.g., using a network **502**) to

14

other machines. In a networked deployment, the machine may operate in the capacity of a server or a client user machine in a server-client user network environment, or as a peer machine in a peer-to-peer (or distributed) network environment.

The machine may comprise a server computer, a client user computer, a personal computer (PC), a tablet, a smart phone, a laptop computer, a desktop computer, a control system, a network router, switch or bridge, or any machine capable of executing a set of instructions (sequential or otherwise) that specify actions to be taken by that machine. It will be understood that a communication device of the subject disclosure includes broadly any electronic device that provides voice, video or data communication. Further, while a single machine is illustrated, the term "machine" shall also be taken to include any collection of machines that individually or jointly execute a set (or multiple sets) of instructions to perform any one or more of the methods discussed herein.

Computer system **500** may include a processor (or controller) **504** (e.g., a central processing unit (CPU)), a graphics processing unit (GPU, or both), a main memory **506** and a static memory **508**, which communicate with each other via a bus **510**. The computer system **500** may further include a display unit **512** (e.g., a liquid crystal display (LCD), a flat panel, or a solid state display). Computer system **500** may include an input device **514** (e.g., a keyboard), a cursor control device **516** (e.g., a mouse), a disk drive unit **518**, a signal generation device **520** (e.g., a speaker or remote control) and a network interface device **522**. In distributed environments, the embodiments described in the subject disclosure can be adapted to utilize multiple display units **512** controlled by two or more computer systems **500**. In this configuration, presentations described by the subject disclosure may in part be shown in a first of display units **512**, while the remaining portion is presented in a second of display units **512**.

The disk drive unit **518** may include a tangible computer-readable storage medium **524** on which is stored one or more sets of instructions (e.g., software **526**) embodying any one or more of the methods or functions described herein, including those methods illustrated above. Instructions **526** may also reside, completely or at least partially, within main memory **506**, static memory **508**, or within processor **504** during execution thereof by the computer system **500**. Main memory **506** and processor **504** also may constitute tangible computer-readable storage media.

As shown in FIG. 6, telecommunication system **600** may include wireless transmit/receive units (WTRUs) **602**, a RAN **604**, a core network **606**, a public switched telephone network (PSTN) **608**, the Internet **610**, or other networks **612**, though it will be appreciated that the disclosed examples contemplate any number of WTRUs, base stations, networks, or network elements. Each WTRU **602** may be any type of device configured to operate or communicate in a wireless environment. For example, a WTRU may comprise drone **102**, a mobile device, network device **300**, or the like, or any combination thereof. By way of example, WTRUs **602** may be configured to transmit or receive wireless signals and may include a UE, a mobile station, a mobile device, a fixed or mobile subscriber unit, a pager, a cellular telephone, a PDA, a smartphone, a laptop, a netbook, a personal computer, a wireless sensor, consumer electronics, or the like. WTRUs **602** may be configured to transmit or receive wireless signals over an air interface **614**.

Telecommunication system **600** may also include one or more base stations **616**. Each of base stations **616** may be

15

any type of device configured to wirelessly interface with at least one of the WTRUs 602 to facilitate access to one or more communication networks, such as core network 606, PSTN 608, Internet 610, or other networks 612. By way of example, base stations 616 may be a base transceiver station (BTS), a Node-B, an eNode B, a Home Node B, a Home eNode B, a site controller, an access point (AP), a wireless router, or the like. While base stations 616 are each depicted as a single element, it will be appreciated that base stations 616 may include any number of interconnected base stations or network elements.

RAN 604 may include one or more base stations 616, along with other network elements, such as a base station controller (BSC), a radio network controller (RNC), or relay nodes. One or more base stations 616 may be configured to transmit or receive wireless signals within a particular geographic region, which may be referred to as a cell. The cell may further be divided into cell sectors. For example, the cell associated with base station 616 may be divided into three sectors such that base station 616 may include three transceivers: one for each sector of the cell. In another example, base station 616 may employ multiple-input multiple-output (MIMO) technology and, therefore, may utilize multiple transceivers for each sector of the cell.

Base stations 616 may communicate with one or more of WTRUs 602 over air interface 614, which may be any suitable wireless communication link (e.g., RF, microwave, infrared (IR), ultraviolet (UV), or visible light). Air interface 614 may be established using any suitable radio access technology (RAT).

More specifically, as noted above, telecommunication system 600 may be a multiple access system and may employ one or more channel access schemes, such as CDMA, TDMA, FDMA, OFDMA, SC-FDMA, or the like. For example, base station 616 in RAN 604 and WTRUs 602 connected to RAN 604 may implement a radio technology such as Universal Mobile Telecommunications System (UMTS) Terrestrial Radio Access (UTRA) that may establish air interface 614 using wideband CDMA (WCDMA). WCDMA may include communication protocols, such as High-Speed Packet Access (HSPA) or Evolved HSPA (HSPA+). HSPA may include High-Speed Downlink Packet Access (HSDPA) or High-Speed Uplink Packet Access (HSUPA).

As another example base station 616 and WTRUs 602 that are connected to RAN 604 may implement a radio technology such as Evolved UMTS Terrestrial Radio Access (E-UTRA), which may establish air interface 614 using LTE or LTE-Advanced (LTE-A).

Optionally base station 616 and WTRUs 602 connected to RAN 604 may implement radio technologies such as IEEE 802.16 (i.e., Worldwide Interoperability for Microwave Access (WiMAX)), CDMA2000, CDMA2000 1x, CDMA2000 EV-DO, Interim Standard 2000 (IS-2000), Interim Standard 95 (IS-95), Interim Standard 856 (IS-856), GSM, Enhanced Data rates for GSM Evolution (EDGE), GSM EDGE (GERAN), or the like.

Base station 616 may be a wireless router, Home Node B, Home eNode B, or access point, for example, and may utilize any suitable RAT for facilitating wireless connectivity in a localized area, such as a place of business, a home, a vehicle, a campus, or the like. For example, base station 616 and associated WTRUs 602 may implement a radio technology such as IEEE 802.11 to establish a wireless local area network (WLAN). As another example, base station 616 and associated WTRUs 602 may implement a radio technology such as IEEE 802.15 to establish a wireless

16

personal area network (WPAN). In yet another example, base station 616 and associated WTRUs 602 may utilize a cellular-based RAT (e.g., WCDMA, CDMA2000, GSM, LTE, LTE-A, etc.) to establish a picocell or femtocell. As shown in FIG. 6, base station 616 may have a direct connection to Internet 610. Thus, base station 616 may not be required to access Internet 610 via core network 606.

RAN 604 may be in communication with core network 606, which may be any type of network configured to provide voice, data, applications, and/or voice over internet protocol (VoIP) services to one or more WTRUs 602. For example, core network 606 may provide call control, billing services, mobile location-based services, pre-paid calling, Internet connectivity, video distribution or high-level security functions, such as user authentication. Although not shown in FIG. 6, it will be appreciated that RAN 604 or core network 606 may be in direct or indirect communication with other RANs that employ the same RAT as RAN 604 or a different RAT. For example, in addition to being connected to RAN 604, which may be utilizing an E-UTRA radio technology, core network 606 may also be in communication with another RAN employing a GSM radio technology.

Core network 606 may also serve as a gateway for WTRUs 602 to access PSTN 608, Internet 610, or other networks 612. PSTN 608 may include circuit-switched telephone networks that provide plain old telephone service (POTS). For LTE core networks, core network 606 may use IMS core 614 to provide access to PSTN 608. Internet 610 may include a global system of interconnected computer networks or devices that use common communication protocols, such as the transmission control protocol (TCP), user datagram protocol (UDP), or IP in the TCP/IP internet protocol suite. Other networks 612 may include wired or wireless communications networks owned or operated by other service providers. For example, other networks 612 may include another core network connected to one or more RANs, which may employ the same RAT as RAN 604 or a different RAT.

Some or all WTRUs 602 in telecommunication system 600 may include multi-mode capabilities. That is, WTRUs 602 may include multiple transceivers for communicating with different wireless networks over different wireless links. For example, one or more WTRUs 602 may be configured to communicate with base station 616, which may employ a cellular-based radio technology, and with base station 616, which may employ an IEEE 802 radio technology.

FIG. 7 is an example system 700 including RAN 604 and core network 606. As noted above, RAN 604 may employ an E-UTRA radio technology to communicate with WTRUs 602 over air interface 614. RAN 604 may also be in communication with core network 606.

RAN 604 may include any number of eNode-Bs 702 while remaining consistent with the disclosed technology. One or more eNode-Bs 702 may include one or more transceivers for communicating with the WTRUs 602 over air interface 614. Optionally, eNode-Bs 702 may implement MIMO technology. Thus, one of eNode-Bs 702, for example, may use multiple antennas to transmit wireless signals to, or receive wireless signals from, one of WTRUs 602.

Each of eNode-Bs 702 may be associated with a particular cell and may be configured to handle radio resource management decisions, handover decisions, scheduling of users in the uplink or downlink, or the like. As shown in FIG. 7 eNode-Bs 702 may communicate with one another over an X2 interface.

Core network **606** shown in FIG. 7 may include a mobility management gateway or entity (MME) **704**, a serving gateway **706**, or a packet data network (PDN) gateway **708**. While each of the foregoing elements are depicted as part of core network **606**, it will be appreciated that any one of these elements may be owned or operated by an entity other than the core network operator.

MME **704** may be connected to each of eNode-Bs **702** in RAN **604** via an S1 interface and may serve as a control node. For example, MME **704** may be responsible for authenticating users of WTRUs **602**, bearer activation or deactivation, selecting a particular serving gateway during an initial attach of WTRUs **602**, or the like. MME **704** may also provide a control plane function for switching between RAN **604** and other RANs that employ other radio technologies, such as GSM or WCDMA.

Serving gateway **706** may be connected to each of eNode-Bs **702** in RAN **604** via the S1 interface. Serving gateway **706** may generally route or forward user data packets to or from the WTRUs **602**. Serving gateway **706** may also perform other functions, such as anchoring user planes during inter-eNode B handovers, triggering paging when downlink data is available for WTRUs **602**, managing or storing contexts of WTRUs **602**, or the like.

Serving gateway **706** may also be connected to PDN gateway **708**, which may provide WTRUs **602** with access to packet-switched networks, such as Internet **610**, to facilitate communications between WTRUs **602** and IP-enabled devices.

Core network **606** may facilitate communications with other networks. For example, core network **606** may provide WTRUs **602** with access to circuit-switched networks, such as PSTN **608**, such as through IMS core **614**, to facilitate communications between WTRUs **602** and traditional land-line communications devices. In addition, core network **606** may provide the WTRUs **602** with access to other networks **612**, which may include other wired or wireless networks that are owned or operated by other service providers.

FIG. 8 depicts an overall block diagram of an example packet-based mobile cellular network environment, such as a GPRS network as described herein. In the example packet-based mobile cellular network environment shown in FIG. 8, there are a plurality of base station subsystems (BSS) **800** (only one is shown), each of which comprises a base station controller (BSC) **802** serving a plurality of BTSs, such as BTSs **804**, **806**, **808**. BTSs **804**, **806**, **808** are the access points where users of packet-based mobile devices become connected to the wireless network. In example fashion, the packet traffic originating from mobile devices is transported via an over-the-air interface to BTS **808**, and from BTS **808** to BSC **802**. Base station subsystems, such as BSS **800**, are a part of internal frame relay network **810** that can include a service GPRS support nodes (SGSN), such as SGSN **812** or SGSN **814**. Each SGSN **812**, **814** is connected to an internal packet network **816** through which SGSN **812**, **814** can route data packets to or from a plurality of gateway GPRS support nodes (GGSN) **818**, **820**, **822**. As illustrated, SGSN **814** and GGSNs **818**, **820**, **822** are part of internal packet network **816**. GGSNs **818**, **820**, **822** mainly provide an interface to external IP networks such as PLMN **824**, corporate intranets/internets **826**, or Fixed-End System (FES) or the public Internet **828**. As illustrated, subscriber corporate network **826** may be connected to GGSN **820** via a firewall **830**. PLMN **824** may be connected to GGSN **820** via a border gateway router (BGR) **832**. A Remote Authen-

tication Dial-In User Service (RADIUS) server **834** may be used for caller authentication when a user calls corporate network **826**.

Generally, there may be a several cell sizes in a network, referred to as macro, micro, pico, femto or umbrella cells. The coverage area of each cell is different in different environments. Macro cells can be regarded as cells in which the base station antenna is installed in a mast or a building above average roof top level. Micro cells are cells whose antenna height is under average roof top level. Micro cells are typically used in urban areas. Pico cells are small cells having a diameter of a few dozen meters. Pico cells are used mainly indoors. Femto cells have the same size as pico cells, but a smaller transport capacity. Femto cells are used indoors, in residential or small business environments. On the other hand, umbrella cells are used to cover shadowed regions of smaller cells and fill in gaps in coverage between those cells.

FIG. 9 illustrates an architecture of a typical GPRS network **900** as described herein. The architecture depicted in FIG. 9 may be segmented into four groups: users **902**, RAN **904**, core network **906**, and interconnect network **908**. Users **902** comprise a plurality of end users, who each may use one or more devices **910**. Note that device **910** is referred to as a mobile subscriber (MS) in the description of network shown in FIG. 9. In an example, device **910** comprises a communications device (e.g., mobile device **102**, mobile positioning center **116**, network device **300**, any of detected devices **500**, second device **508**, access device **604**, access device **606**, access device **608**, access device **610** or the like, or any combination thereof). Radio access network **904** comprises a plurality of BSSs such as BSS **912**, which includes a BTS **914** and a BSC **916**. Core network **906** may include a host of various network elements. As illustrated in FIG. 9, core network **906** may comprise MSC **918**, service control point (SCP) **920**, gateway MSC (GMSC) **922**, SGSN **924**, home location register (HLR) **926**, authentication center (AuC) **928**, domain name system (DNS) server **930**, and GGSN **932**. Interconnect network **908** may also comprise a host of various networks or other network elements. As illustrated in FIG. 9, interconnect network **908** comprises a PSTN **934**, an FES/Internet **936**, a firewall **1038** (FIG. 10), or a corporate network **940**.

An MSC can be connected to a large number of BSCs. At MSC **918**, for instance, depending on the type of traffic, the traffic may be separated in that voice may be sent to PSTN **934** through GMSC **922**, or data may be sent to SGSN **924**, which then sends the data traffic to GGSN **932** for further forwarding.

When MSC **918** receives call traffic, for example, from BSC **916**, it sends a query to a database hosted by SCP **920**, which processes the request and issues a response to MSC **918** so that it may continue call processing as appropriate.

HLR **926** is a centralized database for users to register to the GPRS network. HLR **926** stores static information about the subscribers such as the International Mobile Subscriber Identity (IMSI), subscribed services, or a key for authenticating the subscriber. HLR **926** also stores dynamic subscriber information such as the current location of the MS. Associated with HLR **926** is AuC **928**, which is a database that contains the algorithms for authenticating subscribers and includes the associated keys for encryption to safeguard the user input for authentication.

In the following, depending on context, "mobile subscriber" or "MS" sometimes refers to the end user and sometimes to the actual portable device, such as a mobile device, used by an end user of the mobile cellular service.

When a mobile subscriber turns on his or her mobile device, the mobile device goes through an attach process by which the mobile device attaches to an SGSN of the GPRS network. In FIG. 9, when MS 910 initiates the attach process by turning on the network capabilities of the mobile device, an attach request is sent by MS 910 to SGSN 924. The SGSN 924 queries another SGSN, to which MS 910 was attached before, for the identity of MS 910. Upon receiving the identity of MS 910 from the other SGSN, SGSN 924 requests more information from MS 910. This information is used to authenticate MS 910 together with the information provided by HLR 926. Once verified, SGSN 924 sends a location update to HLR 926 indicating the change of location to a new SGSN, in this case SGSN 924. HLR 926 notifies the old SGSN, to which MS 910 was attached before, to cancel the location process for MS 910. HLR 926 then notifies SGSN 924 that the location update has been performed. At this time, SGSN 924 sends an Attach Accept message to MS 910, which in turn sends an Attach Complete message to SGSN 924.

Next, MS 910 establishes a user session with the destination network, corporate network 940, by going through a Packet Data Protocol (PDP) activation process. Briefly, in the process, MS 910 requests access to the Access Point Name (APN), for example, UPS.com, and SGSN 924 receives the activation request from MS 910. SGSN 924 then initiates a DNS query to learn which GGSN 932 has access to the UPS.com APN. The DNS query is sent to a DNS server within core network 906, such as DNS server 930, which is provisioned to map to one or more GGSNs in core network 906. Based on the APN, the mapped GGSN 932 can access requested corporate network 940. SGSN 924 then sends to GGSN 932 a Create PDP Context Request message that contains necessary information. GGSN 932 sends a Create PDP Context Response message to SGSN 924, which then sends an Activate PDP Context Accept message to MS 910.

Once activated, data packets of the call made by MS 910 can then go through RAN 904, core network 906, and interconnect network 908, in a particular FES/ Internet 936 and firewall 1038, to reach corporate network 940.

FIG. 10 illustrates a block diagram of an example PLMN architecture that may be replaced by a telecommunications system. In FIG. 10, solid lines may represent user traffic signals, and dashed lines may represent support signaling. MS 1002 is the physical equipment used by the PLMN subscriber. For example, drone 102, network device 300, the like, or any combination thereof may serve as MS 1002. MS 1002 may be one of, but not limited to, a cellular telephone, a cellular telephone in combination with another electronic device or any other wireless mobile communication device.

MS 1002 may communicate wirelessly with BSS 1004. BSS 1004 contains BSC 1006 and a BTS 1008. BSS 1004 may include a single BSC 1006/BTS 1008 pair (base station) or a system of BSC/BTS pairs that are part of a larger network. BSS 1004 is responsible for communicating with MS 1002 and may support one or more cells. BSS 1004 is responsible for handling cellular traffic and signaling between MS 1002 and a core network 1010. Typically, BSS 1004 performs functions that include, but are not limited to, digital conversion of speech channels, allocation of channels to mobile devices, paging, or transmission/reception of cellular signals.

Additionally, MS 1002 may communicate wirelessly with RNS 1012. RNS 1012 contains a Radio Network Controller (RNC) 1014 and one or more Nodes B 1016. RNS 1012 may support one or more cells. RNS 1012 may also include one

or more RNC 1014/Node B 1016 pairs or alternatively a single RNC 1014 may manage multiple Nodes B 1016. RNS 1012 is responsible for communicating with MS 1002 in its geographically defined area. RNC 1014 is responsible for controlling Nodes B 1016 that are connected to it and is a control element in a UMTS radio access network. RNC 1014 performs functions such as, but not limited to, load control, packet scheduling, handover control, security functions, or controlling MS 1002 access to core network 1010.

An E-UTRA Network (E-UTRAN) 1018 is a RAN that provides wireless data communications for MS 1002 and UE 1024. E-UTRAN 1018 provides higher data rates than traditional UMTS. It is part of the LTE upgrade for mobile networks, and later releases meet the requirements of the International Mobile Telecommunications (IMT) Advanced and are commonly known as a 4G networks. E-UTRAN 1018 may include of series of logical network components such as E-UTRAN Node B (eNB) 1020 and E-UTRAN Node B (eNB) 1022. E-UTRAN 1018 may contain one or more eNBs. User equipment (UE) 1024 may be any mobile device capable of connecting to E-UTRAN 1018 including, but not limited to, a personal computer, laptop, mobile device, wireless router, or other device capable of wireless connectivity to E-UTRAN 1018. The improved performance of the E-UTRAN 1018 relative to a typical UMTS network allows for increased bandwidth, spectral efficiency, and functionality including, but not limited to, voice, high-speed applications, large data transfer or IPTV, while still allowing for full mobility.

Typically MS 1002 may communicate with any or all of BSS 1004, RNS 1012, or E-UTRAN 1018. In a illustrative system, each of BSS 1004, RNS 1012, and E-UTRAN 1018 may provide MS 1002 with access to core network 1010. Core network 1010 may include of a series of devices that route data and communications between end users. Core network 1010 may provide network service functions to users in the circuit switched (CS) domain or the packet switched (PS) domain. The CS domain refers to connections in which dedicated network resources are allocated at the time of connection establishment and then released when the connection is terminated. The PS domain refers to communications and data transfers that make use of autonomous groupings of bits called packets. Each packet may be routed, manipulated, processed or handled independently of all other packets in the PS domain and does not require dedicated network resources.

The circuit-switched MGW function (CS-MGW) 1026 is part of core network 1010, and interacts with VLR/MSC server 1028 and GMSC server 1030 in order to facilitate core network 1010 resource control in the CS domain. Functions of CS-MGW 1026 include, but are not limited to, media conversion, bearer control, payload processing or other mobile network processing such as handover or anchoring. CS-MGW 1026 may receive connections to MS 1002 through BSS 1004 or RNS 1012.

SGSN 1032 stores subscriber data regarding MS 1002 in order to facilitate network functionality. SGSN 1032 may store subscription information such as, but not limited to, the IMSI, temporary identities, or PDP addresses. SGSN 1032 may also store location information such as, but not limited to, GGSN address for each GGSN 1034 where an active PDP exists. GGSN 1034 may implement a location register function to store subscriber data it receives from SGSN 1032 such as subscription or location information.

Serving gateway (S-GW) 1036 is an interface which provides connectivity between E-UTRAN 1018 and core network 1010. Functions of S-GW 1036 include, but are not

limited to, packet routing, packet forwarding, transport level packet processing, or user plane mobility anchoring for inter-network mobility. PCRF **1038** uses information gathered from P-GW **1036**, as well as other sources, to make applicable policy and charging decisions related to data flows, network resources or other network administration functions. PDN gateway (PDN-GW) **1040** may provide user-to-services connectivity functionality including, but not limited to, GPRS/EPC network anchoring, bearer session anchoring and control, or IP address allocation for PS domain connections.

HSS **1042** is a database for user information and stores subscription data regarding MS **1002** or UE **1024** for handling calls or data sessions. Networks may contain one HSS **1042** or more if additional resources are required. Example data stored by HSS **1042** include, but is not limited to, user identification, numbering or addressing information, security information, or location information. HSS **1042** may also provide call or session establishment procedures in both the PS and CS domains.

VLR/MSC Server **1028** provides user location functionality. When MS **1002** enters a new network location, it begins a registration procedure. A MSC server for that location transfers the location information to the VLR for the area. A VLR and MSC server may be located in the same computing environment, as is shown by VLR/MSC server **1028**, or alternatively may be located in separate computing environments. A VLR may contain, but is not limited to, user information such as the IMSI, the Temporary Mobile Station Identity (TMSI), the Local Mobile Station Identity (LMSI), the last known location of the mobile station, or the SGSN where the mobile station was previously registered. The MSC server may contain information such as, but not limited to, procedures for MS **1002** registration or procedures for handover of MS **1002** to a different section of core network **1010**. GMSC server **1030** may serve as a connection to alternate GMSC servers for other MSs in larger networks.

EIR **1044** is a logical element which may store the IMEI for MS **1002**. User equipment may be classified as either “white listed” or “black listed” depending on its status in the network. If MS **1002** is stolen and put to use by an unauthorized user, it may be registered as “black listed” in EIR **1044**, preventing its use on the network. A MME **1046** is a control node which may track MS **1002** or UE **1024** if the devices are idle. Additional functionality may include the ability of MME **1046** to contact idle MS **1002** or UE **1024** if retransmission of a previous session is required.

As described herein, a telecommunications system wherein management and control utilizing a software designed network (SDN) and a simple IP are based, at least in part, on user equipment, may provide a wireless management and control framework that enables common wireless management and control, such as mobility management, radio resource management, QoS, load balancing, etc., across many wireless technologies, e.g. LTE, Wi-Fi, and future 5G access technologies; decoupling the mobility control from data planes to let them evolve and scale independently; reducing network state maintained in the network based on user equipment types to reduce network cost and allow massive scale; shortening cycle time and improving network upgradability; flexibility in creating end-to-end services based on types of user equipment and applications, thus improve customer experience; or improving user equipment power efficiency and battery life—especially for simple M2M devices—through enhanced wireless management.

While examples of a telecommunications system in which emergency alerts can be processed and managed have been described in connection with various computing devices/processors, the underlying concepts may be applied to any computing device, processor, or system capable of facilitating a telecommunications system. The various techniques described herein may be implemented in connection with hardware or software or, where appropriate, with a combination of both. Thus, the methods and devices may take the form of program code (i.e., instructions) embodied in concrete, tangible, storage media having a concrete, tangible, physical structure. Examples of tangible storage media include floppy diskettes, CD-ROMs, DVDs, hard drives, or any other tangible machine-readable storage medium (computer-readable storage medium). Thus, a computer-readable storage medium is not a signal. A computer-readable storage medium is not a transient signal. Further, a computer-readable storage medium is not a propagating signal. A computer-readable storage medium as described herein is an article of manufacture. When the program code is loaded into and executed by a machine, such as a computer, the machine becomes a device for telecommunications. In the case of program code execution on programmable computers, the computing device will generally include a processor, a storage medium readable by the processor (including volatile or nonvolatile memory or storage elements), at least one input device, and at least one output device. The program(s) can be implemented in assembly or machine language, if desired. The language can be a compiled or interpreted language, and may be combined with hardware implementations.

The methods and devices associated with a telecommunications system as described herein also may be practiced via communications embodied in the form of program code that is transmitted over some transmission medium, such as over electrical wiring or cabling, through fiber optics, or via any other form of transmission, wherein, when the program code is received and loaded into and executed by a machine, such as an EPROM, a gate array, a programmable logic device (PLD), a client computer, or the like, the machine becomes an device for implementing telecommunications as described herein. When implemented on a general-purpose processor, the program code combines with the processor to provide a unique device that operates to invoke the functionality of a telecommunications system.

EXAMPLES

Example 1. A network device comprising: a processor, an input/output device coupled to the processor, and a memory coupled with the processor, the memory comprising executable instructions that when executed by the processor cause the processor to effectuate operations including instantiating at least one node comprising a packet processor and a network interface, the packet processor configured to process a packet header at a network layer, wherein the at least one node includes a common configuration; extracting virtual network function parameters through an inference engine; generating a virtual network function template based on the virtual network function parameters, wherein the virtual network function template instantiates at least one virtual network function by assembling the at least one virtual network function from the at least one node; and automatically configures the virtual network function for onboarding onto a platform.

Example 2. The network device of example 1, wherein the virtual network function template includes instructions that

23

when executed by the processor cause the processor to effectuate operations comprising: generating a configuration file, the configuration file generating onboarding metadata for at least one network level; creating a virtual machine configuration script; and wherein the configuration file includes a script interacting with an orchestration platform to validate the at least one node.

Example 3. The network device of example 2, wherein creating the virtual machine configuration script includes specifying a virtual NIC and at least one configuration parameter corresponding to the virtual NIC.

Example 4. The network device of example 1, the virtual network function parameters including a packet processing graph, wherein the packet processing graph includes at least one of a protocol, a packet processing parameter, a dependency, and a metric.

Example 5. The network device of example 4, wherein the protocol includes at least one of an internet protocol level information, a Mac information, and a routing table; wherein the packet processing parameter includes at least one of a hardware requirement and a software requirement.

Example 6. The network device of example 1, wherein the platform is an ECOMP platform.

Example 7. The network device of example 1, wherein the packet processor is a VPP.

Example 8. The network device of example 1, wherein a function information is communicated to the inference engine from the input/output device, the function information including at least one of a hosting requirement, an application logic, a TCP/IP building block and an Ethernet building block.

Example 9. The network device of example 1, wherein the inference engine generates an artifact including at least one of a virtual machine configuration, a container configuration, a firewall configuration, an IP layer configuration, and an Ethernet configuration.

Example 10. The network device of example 1, wherein the virtual network function template includes at least one of a HEAT template; a CLI for at least one of a firewall, an executable, an SDN controller, and an API; a Linux IP or GW startup script; a KVM domain XML; and a NIC.

Example 11. A system for creating a virtual network function comprising: an inference engine in communication with an input device configured to receive a virtual network function input; the inference engine configured to extract at least one virtual network function parameter from the virtual network function input to define a virtual network function template; the virtual network function template comprising at least one of a configuration file and a configuration script, and a packet processing graph; and a builder module, the builder module configured to assemble at least one virtual network function from at least one node according to the virtual network function template and configure the at least one node for onboarding based on the virtual network function template.

Example 12. The system of example 1 further comprising a unified virtual network function infrastructure having a virtual packet processor, wherein the at least one node resides within the unified virtual network function infrastructure, the at least one node having a common configuration.

Example 13. The system of example 11 further comprising a platform, wherein the virtual network function created by the builder module interacts with the platform based on the virtual network function template.

Example 14. The system of example 13, wherein the platform is an ECOMP platform.

24

Example 15. The system of example 11, wherein the packet processing graph includes at least one of a protocol, a packet processing parameter, a dependency, and a metric.

Example 16. The system of example 15, wherein the protocol includes at least one of an internet protocol level information, a Mac information, and a routing table; wherein the packet processing parameter includes at least one of a hardware requirement and a software requirement.

Example 17. The system of example 11, wherein the configuration file includes metadata, wherein the metadata includes at least one of a platform template, a local configuration script, an external configuration script, and a platform interface.

Example 18. A computer readable storage medium comprising executable instructions that cause a processor to effectuate operations comprising: instantiating at least one node comprising a packet processor and a network interface, the packet processor configured to process a packet header at a network layer, wherein the at least one node includes a common configuration; extracting virtual network function parameters through an inference engine; generating a virtual network function template based on the virtual network function parameters, wherein the virtual network function template instantiates at least one virtual network function by assembling the at least one virtual network function from the at least one node; and automatically configures the virtual network function for onboarding onto a platform.

Example 19. The computer readable media further effectuating operations comprising: generating a configuration file, the configuration file generating onboarding metadata for at least one network level; creating a virtual machine configuration script; and wherein the configuration file includes a script interacting with an orchestration platform to validate the at least one node.

The invention claimed is:

1. A network device comprising:

a processing system including a processor; and
a memory that stores executable instructions that, when executed by the processing system, facilitate performance of operations, the operations comprising:

identifying a common code structure of a plurality of nodes having a common configuration that includes common protocol stacks;

parsing function information contained in the common code structure to obtain parsed function information; extracting, from the parsed function information, virtual network function parameters;

generating a virtual network function template based on the virtual network function parameters and using the common protocol stacks, wherein the virtual network function template instantiates at least one virtual network function by assembling the at least one virtual network function from at least one node of the plurality of nodes;

generating a configuration file according to the virtual network function template; and

generating a virtual machine configuration script according to the virtual network function template, the configuration file and the virtual machine configuration script are configured for interaction with an orchestration platform to facilitate an automatic configuration of the at least one virtual network function according to the virtual network function template for onboarding onto a platform, wherein the onboarding onto the platform is based on a function input designating a desired function or group of functions for the at least one virtual network function,

25

wherein the identifying the common code structure further comprises reading functions of the at least one virtual network function, wherein the configuration file comprises onboarding metadata used for onboarding the metadata for at least one network level, wherein the metadata comprises a local configuration script, an external configuration script and an interface to the platform, and wherein the configuration file comprises a script configured for interacting with the orchestration platform to validate the at least one node.

2. The network device of claim 1, wherein the generating the virtual machine configuration script further comprises specifying a virtual network interface card (NIC) and at least one configuration parameter corresponding to the virtual NIC, and wherein the desired function or group of functions comprises a type of function and a layer at which the at least one virtual network function is implemented.

3. The network device of claim 1, wherein the virtual network function parameters comprise a packet processing graph of nodes, wherein the parsing function information comprises parsing function information contained in the packet processing graph, and wherein the packet processing graph comprises at least one of a protocol, a packet processing parameter, a dependency, and a metric.

4. The network device of claim 3, wherein the protocol includes at least one of an internet protocol level information, media access control (MAC) information, and a routing table, wherein the packet processing parameter includes at least one of a hardware requirement and a software requirement.

5. The network device of claim 1, wherein the platform is an enhanced control, orchestration, management, and policy (ECOMP) platform.

6. The network device of claim 1, wherein the at least one node comprises a packet processor configured to process a packet header at a network layer, and wherein the virtual network function template comprises a heat template.

7. The network device of claim 1, wherein the common code structure is obtained from function information obtained via an input/output device, the function information including at least one of a hosting requirement, an application logic, a TCP/IP building block and an Ethernet building block.

8. The network device of claim 1, wherein the virtual network function parameters are extracted by an inference engine adapted to generate an artifact including at least one of a virtual machine configuration, a container configuration, a firewall configuration, an IP layer configuration, and an Ethernet configuration.

9. The network device of claim 1, wherein the virtual network function template includes at least one of a Command Line Interface (CLI) for at least one of a firewall, an executable, a software defined network (SDN) controller, and an application programming interface (API), a Linux internet protocol (IP), a Kernel Virtual Machine (KVM) domain Extensible Markup Language (XML), and a network interface card (NIC).

10. The network device of claim 1, wherein the operations further comprise:

instantiating a network interface, wherein the network interface is used to generate the configuration file and the virtual machine configuration script at each layer, and wherein the virtual network function template comprises a heat template.

26

11. A method comprising:

determining, by a processing system including a processor, a common code structure of a plurality of nodes in a network having a common configuration that includes common protocol stacks;

parsing function information contained in the common code structure to obtain parsed function information;

obtaining, by the processing system and from the parsed function information, virtual network function parameters through an inference engine;

generating, by the processing system, a virtual network function template based on the virtual network function parameters and using the common protocol stacks, wherein the virtual network function template instantiates at least one virtual network function by assembling the at least one virtual network function from at least one node of the plurality of nodes;

generating, by the processing system, a configuration file according to the virtual network function template; and

generating, by the processing system, a virtual machine configuration script according to the virtual network function template, the configuration file and the virtual machine configuration script are configured for interaction with an orchestration platform to facilitate an automatic configuration of the at least one virtual network function according to the virtual network function template for onboarding onto a platform, wherein the onboarding onto the platform is based on a function input designating a desired function or group of functions for the at least one virtual network function,

wherein the determining the common code structure further comprises reading functions of the at least one virtual network function, wherein the configuration file comprises onboarding metadata used for onboarding the metadata for at least one network level, wherein the metadata comprises a local configuration script, an external configuration script and an interface to the platform, and wherein the configuration file comprises a script configured for interacting with the orchestration platform to validate the at least one node.

12. The method of claim 11, wherein the generating the virtual machine configuration script further comprises includes specifying a virtual network interface card (NIC) and at least one configuration parameter corresponding to the virtual NIC.

13. The method of claim 11, wherein the virtual network function parameters comprise a packet processing graph, and wherein the packet processing graph comprises at least one of a protocol, a packet processing parameter, a dependency, and a metric.

14. The method of claim 13, wherein the protocol includes at least one of an internet protocol level information, media access control (MAC) information, and a routing table, and wherein the packet processing parameter includes at least one of a hardware requirement and a software requirement.

15. The method of claim 11, wherein the common code structure is obtained from function information extracted by the inference engine from an input/output device, the function information including at least one of a hosting requirement, an application logic, a TCP/IP building block and an Ethernet building block.

16. The method of claim 11, wherein the inference engine generates an artifact including at least one of a virtual machine configuration, a container configuration, a firewall configuration, an IP layer configuration, and an Ethernet configuration.

27

17. A non-transitory, machine-readable storage medium comprising executable instructions that cause a processor to effectuate operations comprising:

determining a common code structure of a plurality of nodes having a common configuration that includes common protocol stacks; 5

parsing function information contained in the common code structure to obtain parsed function information; deriving, from the parsed function information, virtual network function parameters; 10

generating a virtual network function template based on the virtual network function parameters and using the common protocol stacks, wherein the virtual network function template instantiates at least one virtual network function by assembling the at least one virtual network function from at least one node of the plurality of nodes; 15

generating a configuration file according to the virtual network function template; and

generating a virtual machine configuration script according to the virtual network function template, the con-

28

figuration file and the virtual machine configuration script are configured for interaction with an orchestration platform to facilitate an automatic configuration of the at least one virtual network function according to the virtual network function template for onboarding onto a platform, wherein the onboarding onto the platform is based on a function input designating a desired function or group of functions for the at least one virtual network function,

wherein the determining the common code structure further comprises reading functions of the at least one virtual network function, wherein the configuration file comprises onboarding metadata used for onboarding the metadata for at least one network level, wherein the metadata comprises a local configuration script, an external configuration script and an interface to the platform, and wherein the configuration file comprises a script configured for interacting with the orchestration platform to validate the at least one node.

* * * * *